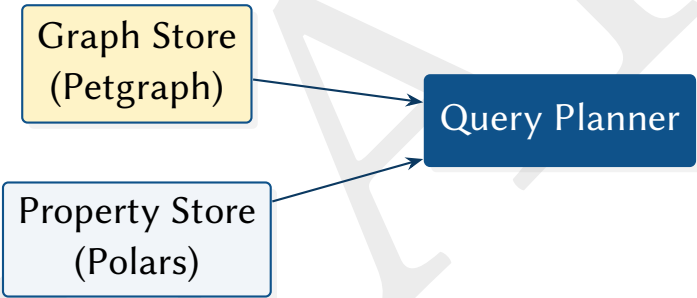


NTE: Network Topology Engine

A Hybrid Graph-Relational Model for Hyper-Scale Synthesis

Simon Knight, Ph.D.
Principal Architect, AutoNetKit Research
March 2026 • Version 2.3.0



Abstract. NTE (Network Topology Engine) is a Rust-based hybrid graph-relational engine for network topology modelling, exposed to Python via PyO3 bindings. Its central design principle is a dual-write store: topology structure lives in a `petgraph::StableDiGraph` for $O(1)$ -hop traversal, whilst node and edge properties are simultaneously maintained in Polars Arrow-backed DataFrames for SIMD-accelerated columnar scans. This report documents the storage architecture, the filter-first query planner, and the "Correctness-by-Construction" policy engine that underpins the AutoNetKit synthesis pipeline.

Version	Date	Changes
2.3.0	March 2026	Integrated v2.3 Ph.D. Insights & Design Rules.
0.3.0	March 2026	Integrated ACI Isolation Levels and CSR Persistence
0.2.0	Jan 2026	GPU-accelerated SSSP and Label Propagation
0.1.0	Nov 2025	Initial Specification and Core Data Model

Todo list

DRAFT

Contents

List of Design Decisions & Rules	2
1 Introduction	3
1.1 What NTE Solves	3
1.2 Why a Bespoke Engine?	4
1.3 Ecosystem: ank_pydantic and ank_netcfg	4
1.4 Who Should Read This Document	5
1.5 How to Read This Document	5
2 Data Model	5
2.1 Formal Definition	5
2.2 Whiteboard-to-Plan (W2P) Strategy	6
2.3 Node Types	6
2.4 Edge Kinds	7
2.4.1 Inter (Connectivity)	7
2.4.2 Intra (Structural / Ownership)	7
2.4.3 Intranode	7
2.4.4 Device Shortcut (DeviceLink)	7
2.5 The Three-Hop Connectivity Pattern	7
2.6 Layer Hierarchy	8
2.7 Multi-Layer Topology Invariants	8
2.8 State Reconciliation: Intent vs. Observed	9
2.9 Property Maps	9
2.10 Workspace Layout	9
3 Storage Architecture	11
3.1 Rationale for Dual-Write	11
3.2 petgraph StableDiGraph	11
3.3 Polars Columnar Store	12
3.4 Dual-Write Consistency Protocol	13
3.5 EventStore Ring Buffer	14
3.6 Pluggable Backends	14
4 Query System	15
4.1 Design Goals	15
4.2 Query Representations	15
4.3 Query Lowering and Execution	15
4.4 Query Complexity: SIMD Pruning vs. Adjacency Traversal	16
4.5 QuerySpec: The Relational Path	16
4.6 Pattern Matching	17
4.7 Selectivity Ranking in Detail	17
4.8 Query Profiling	17
4.9 Plan Cache	17
4.10 Query API Decision Matrix	18
4.11 NTE-QL String Interface	18
4.12 The ExprNode AST	19
4.13 Python Fluent Query Builder	20
4.14 Query Plan Visualisation	21

4.15	NTE-QL Interactive REPL	21
5	End-to-End Query Trace	22
6	Transactions	22
6.1	ACI (Memory-Only) Transaction Model	22
6.2	Isolation Levels	22
6.3	Transaction Snapshots	23
6.4	Python Transaction API	23
6.5	Conflict Detection	24
7	Persistence and Snapshots	24
7.1	Named Snapshots	24
7.2	File Serialisation	25
7.3	CSR Serialisation for Archival	25
7.4	Memory-Mapped Loading (Out-of-Core Execution)	25
7.5	Use Cases for Historical Querying	26
8	GPU Acceleration	26
8.1	Architecture Overview	26
8.2	Hardware Acceleration State	27
8.3	SSSP Kernel	27
8.4	Connected Components Kernel	28
8.5	CPU Fallback	28
8.6	When to Use GPU Acceleration	28
8.7	Invoking GPU Algorithms from Python	29
9	Graph Algorithm Plugins	29
9.1	Plugin Architecture	29
9.2	K-Shortest Paths	30
9.3	Single Point of Failure Detection	30
9.4	Domain-Specific Graph Kernels	30
10	Policy Engine	30
10.1	Design Rationale	30
10.2	Policy File Format	31
10.3	The Starlark Policy Engine	31
10.4	Policy Evaluation	32
10.5	Shadow Topologies and DeltaNet Validation	32
10.6	DeltaNet: Incremental Policy Validation	33
10.7	Worked Policy Examples	33
10.7.1	Transit Redundancy	33
10.7.2	Peer Diversity	33
10.7.3	No Single Point of Failure	34
11	Reactive Events	34
11.1	Event Lifecycle and the Threading Footgun	34
11.2	Closed-Loop Automation Cycle	35
11.3	Event Types	36
11.4	Python Subscription API	36
11.5	Subscription Patterns	37

11.6	Use Cases.....	37
12	Production Diagnostics	38
12.1	Health and Memory Endpoints.....	38
12.2	Chaos Engineering Endpoints	38
12.3	TUI Diagnostics Dashboard	38
13	Distributed Deployment	39
13.1	When to Use Clustering	39
13.2	Raft Consensus via OpenRaft	39
13.3	Configuration	41
13.4	Leader and Follower Roles	41
13.5	Snapshot Transfer	41
13.6	Security and Multi-Tenancy	41
14	Limitations and Future Work	42
14.1	Current Limitations.....	42
14.2	Recently Completed Improvements	43
14.3	Planned Improvements	43
14.4	Vision: Correctness-by-Construction via Formal Verification.....	44
14.5	Performance and Benchmarks	44

List of Figures

1	The Synthesis Funnel: Mapping Abstract Blueprint Intent to High-Fidelity Vendor Artifacts through the NTE Engine and formal Design Rules.....	3
2	The W2P Isomorphism: Mapping Visual Intent to Formal Blueprint.....	6
3	The three-hop connectivity pattern and the automatic DeviceLink shortcut. Intra edges run <i>from</i> endpoint to device; Inter edges are bidirectional.....	8
4	The 3D layered topology model. Logical overlays (BGP, IP) project down onto their physical underlays via structural dependencies.....	8
5	Crate architecture and data flow. Solid arrows show dependency/call direction; the dashed arrow represents the dual-write consistency protocol between the columnar and graph stores.	10
6	Dual-write architecture synergy. Structural queries target the graph engine, while property-heavy predicates target the columnar store.....	11
7	The Wide-Column Sparsity Matrix. Heterogeneous JSON properties are flattened into a single wide DataFrame. Arrow's validity bitmaps efficiently compress the resulting Null regions.	12
8	Hardware-Level Predicate Evaluation. A single AVX-512 instruction evaluates multiple property values in parallel, producing a bitmask that drives the subsequent graph traversal filter.	13
9	The dual-write transaction lifecycle. Copy-on-Write semantics allow $O(1)$ rollback if either the graph or the DataFrame mutation fails.....	13
10	Trait-Based Abstraction. The core query engine interacts only with the TopologyBackend interface, allowing the physical columnar storage (Polars vs. DuckDB) to be swapped at compile time.....	14
11	The Query Lowering Pipeline. High-level intent is decomposed into parallel relational and structural execution tasks.....	15

12	The Cardinality Funnel (Filter-First Execution). Cheap columnar operations aggressively prune candidates, ensuring the more expensive graph traversal engine only visits a tiny fraction of the topology.	16
13	The NTE-QL compilation pipeline. String queries are parsed, converted into programmatic QuerySpec structures, and fed into the same optimizer as the Python APIs.	18
14	The Filter-First Pipeline. Relational scans prune the search space before expensive graph traversals are initiated.	22
15	Serializable isolation conflict detection. Tx1 fails to commit because its read-set ($A=1$) was invalidated by Tx2's concurrent write.	23
16	The .nte archive format. Structural graph data is compressed via zstd, while DataFrames are serialised directly to columnar Parquet files, all packaged into a standard ZIP archive..	25
17	Mmap CSR Execution. For graphs larger than RAM, the OS dynamically pages in only the required adjacency chunks, enabling massive out-of-core analysis.	26
18	GPU SSSP relaxation. A single WGSL compute shader invocation (thread u) reads u 's distance and concurrently proposes new minimums to all its neighbours using <code>atomicMin</code>	28
19	Graph Algorithm Plugin Architecture. Algorithms are invoked by namespace, operate on the shared graph reference, and uniformly return results as Polars DataFrames.	29
20	Starlark Policy Evaluation. The sandboxed runtime safely evaluates Python-like policy code against the topology state to generate deterministic compliance findings.	31
21	The Hybrid Snapshot Reality. While property DataFrames share memory instantly via CoW pointers, the underlying petgraph adjacency lists must be physically deep-copied into new memory allocations.	33
22	Incremental Validation via Impact Radius. By identifying the nodes physically and logically adjacent to a mutation, NTE avoids the $O(V)$ cost of global policy re-evaluation.	33
23	The Event Callback Danger Zone. Synchronous callbacks hold the engine's global write lock. Future versions will move to a lock-free async queue design.	35
24	The Reactive Event Pipeline. Mutations in the core engine generate events which are buffered and asynchronously dispatched to Python callbacks.	35
25	The Autonomous Closed-Loop Cycle. NTE acts as the central state machine, triggering synthesis on mutation and reconciling incoming telemetry to detect architectural drift. ...	35
26	Mockup of the <code>nte-cli</code> TUI Dashboard. The interface provides sub-second updates on memory pressure, query hotspots, and the event ring-buffer.	39
27	Distributed Raft consensus flow. Mutations are proposed to the leader, replicated to followers, and applied only after reaching a quorum.	40
28	Split-Brain Resolution. In a partition, the isolated Node 3 rejects writes because it cannot replicate logs to a majority quorum. Nodes 1 and 2 continue processing safely.	40
29	StableDiGraph Fragmentation. Deletions leave index holes. Over time, heavy mutation destroys spatial locality in memory, degrading CPU cache performance..	42
30	LeapFrog TrieJoin finding multi-way intersections without intermediate tables.	43
31	Scaling Performance: NTE vs. NetworkX for complex property filters. NTE's Polars-backed SIMD scans maintain linear scaling with a significantly lower constant factor compared to row-based alternatives.	45

List of Tables

1	Architectural Positioning: NTE vs. State-of-the-Art.	4
---	---	---

2	Representative micro-benchmarks (Criterion, 100 samples).	45
---	---	----

DRAFT

NTE	Network Topology Engine	3
------------	--	----------

DRAFT

List of Design Decisions & Rules

1	Design Rule: Dual-Write Consistency	3
2	Design Rule: The W2P Isomorphism	6
3	Design Rule: Cross-Layer Structural Integrity	9
4	Design Rule: Truth Isomorphism	9
5	Design Rule: Zero-Copy Persistence	26
6	Design Rule: Shadow Topology Isolation	31
7	Design Rule: Impact-Aware Validation	33
8	Design Rule: Deterministic State Machine Replication	40
9	Design Rule: Formal Invariant Provenance	44

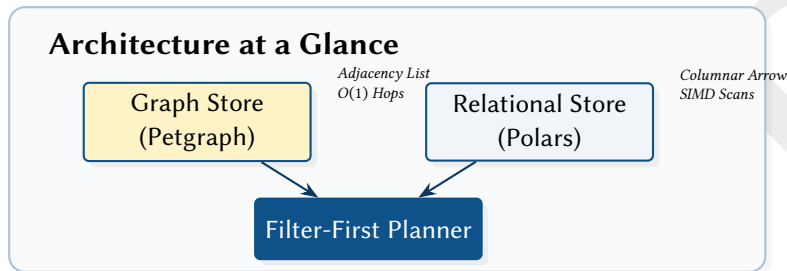
1 Introduction

The scale of modern infrastructure requires a fundamental shift in how topology data is stored and queried.

: Dual-Write Consistency

Every structural mutation to the adjacency graph must be atomically reflected in the columnar property store. This isomorphism ensures that complex queries can be planned using either representation without risk of data divergence.

Insight:
Traditional graph databases excel at structural traversal but fail at columnar analytics; NTE bridges this gap by unifying both representations.



1.1 What NTE Solves

Network automation tools routinely need to answer two very different classes of question about a topology:

1. **Relational / property questions** — “Which routers have AS number 64512 and are in the SYD PoP?” — that are best answered by scanning columns of structured data.
2. **Graph / structural questions** — “Is there a path between device *A* and device *B* using fewer than 4 hops?” — that are best answered by traversing adjacency lists.

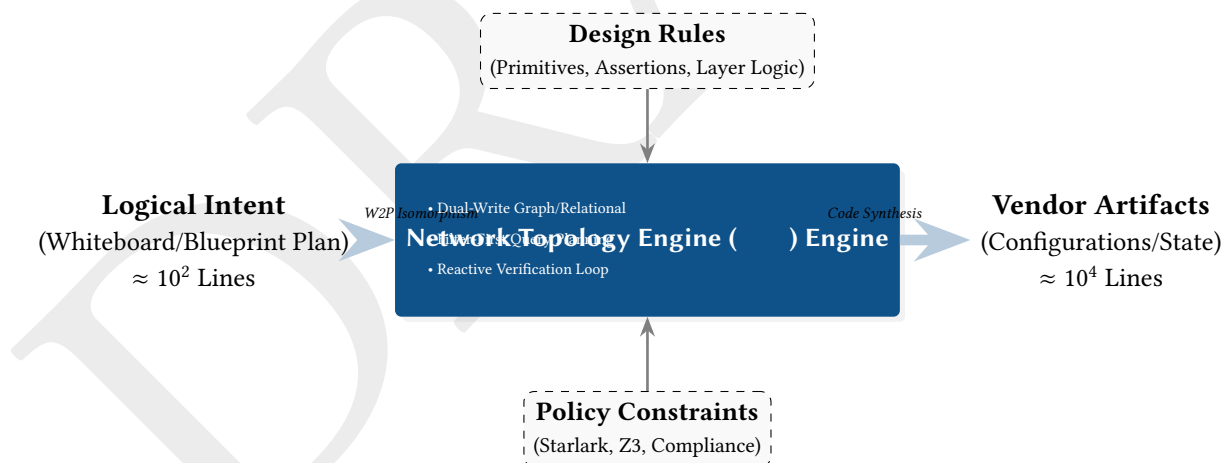


Figure 1. The Synthesis Funnel: Mapping Abstract Blueprint Intent to High-Fidelity Vendor Artifacts through the NTE Engine and formal Design Rules.

Most systems optimise for one and bolt on the other as an afterthought. Relational databases support graph queries via recursive CTEs that are notoriously hard to optimise. Dedicated graph databases (Neo4j [1], KuzuDB [2]) support property lookups but typically store properties row-by-row and lack columnar scan performance.

NTE’s answer is a *dual-write* architecture: every mutation is applied to both a petgraph `StableDiGraph` (for structural queries) and a set of Polars Arrow-backed `DataFrames` (for property scans). A filter-first query planner decides, at runtime, which representation to consult first. In the common case — filter on

a property, then traverse — it runs the cheap columnar scan to get a candidate set of node IDs, then constrains the traversal to those candidates.

1.2 Why a Bespoke Engine?

A natural question is: *Why not use an off-the-shelf graph database like Neo4j, ArangoDB, or Memgraph?* While excellent systems, traditional graph databases impose significant overhead for network automation:

- **Network overhead:** Executing millions of small structural validations via a wire protocol (e.g., Bolt) introduces unacceptable latency. NTE embeds directly in the Python process via PyO3, communicating via zero-copy Arrow memory.
- **Property scan performance:** Most graph DBs store properties row-by-row. When querying “all interfaces with speed > 100G”, they cannot match the SIMD-accelerated columnar scan speeds of Polars.
- **Deterministic compilation:** The `ank_netcfg` compiler requires a strictly deterministic, pure-Rust graph engine to ensure reproducible configuration generation without relying on an external daemon.

Table 1. Architectural Positioning: NTE vs. State-of-the-Art.

Feature	Neo4j	PostgreSQL	Batfish	NTE
Primary Model	Graph (Row)	Relational (Row)	Simulation	Hybrid (SIMD/Graph)
Storage Engine	Native Graph	Heap Tables	In-memory IR	Arrow/CSR Dual-Write
Query Primary	Cypher	SQL (CTE)	Declarative	Filter-First SIMD
FFI Overhead	High (RPC)	High (RPC)	N/A (Tool)	Zero (Embedded PyO3)
"What-If" Speed	Medium (Clone)	Slow (Snapshot)	Fast	Instant (CoW Shadow)

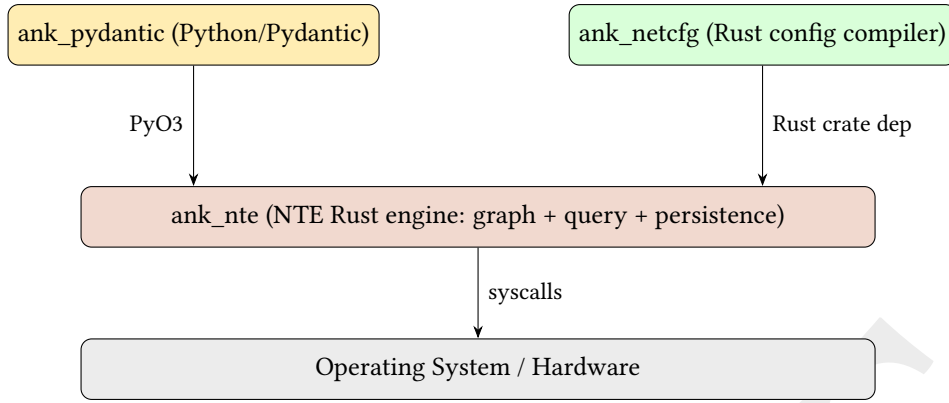
1.3 Ecosystem: `ank_pydantic` and `ank_netcfg`

NTE is a shared backend engine consumed by two alternative frontends:

`ank_pydantic` A Python library that provides high-level, Pydantic-validated network models. It owns the domain schema (what a *Router* looks like, what fields are mandatory) and calls NTE via PyO3 bindings. Suited to interactive exploration, Jupyter notebooks, and Python-centric automation pipelines.

`ank_netcfg` A pure-Rust deterministic network configuration compiler. It consumes `nte-topology` directly (no Python layer), reads declarative YAML blueprints, maps them through a DeviceIR intermediate representation, and renders vendor-neutral device configurations. Suited to CI/CD pipelines, large-scale config generation, and environments where a Python runtime is undesirable.

Both frontends use the same underlying NTE storage, query, and persistence layers. The choice between them is an ergonomic decision, not an architectural one.



All three repositories live side-by-side in the same parent directory.

1.4 Who Should Read This Document

- **Network automation engineers** using `ank_pydantic` or `ank_netcfg` who want to understand what the engine is doing, write performant queries, or interpret error messages from NTE's exception hierarchy.
- **Backend developers** integrating NTE directly (via the Rust API or the Python extension) into larger systems, needing to understand the crate workspace, build process, and extension points.
- **Contributors** wishing to extend the engine with new backends, query operators, or GPU kernels.

1.5 How to Read This Document

Readers wanting to get started quickly should read §2 (Data Model) and the Python API in the User Manual, then refer to individual sections as needed.

§§2–3 cover the core data model and storage engine. §4 covers the query system: the filter-first planner, the fluent Python query builder, NTE-QL, and the interactive REPL. The User Manual provides a Python API walkthrough. §6 covers ACID transactions and conflict detection. §§7–13 cover advanced topics — persistence, GPU acceleration, graph algorithm plugins, policy engine (including shadow topologies), reactive events, production diagnostics, and Raft clustering — that can be read in any order.

2 Data Model

Axiom 1 (Dual-Write Atomicity). *Every mutation to the topology must be reflected atomically in both the graph adjacency store and the columnar property store. A transaction is only committed if both representations are successfully updated.*

[Source](#)

Theorem 1 (Graph-Relational Equivalence). *The set of all nodes V and edges E reachable via graph traversal is equivalent to the set of rows in the primary Polars DataFrames. $\forall v \in \text{DiGraph}, \exists ! \text{row} \in \text{DataFrame}$ mapping to v .*

2.1 Formal Definition

A topology in NTE is a typed, directed, attributed multi-graph $G = (V, E, \ell_V, \ell_E, \pi_V, \pi_E, L, \lambda)$ where:

$V = V_{\text{node}} \cup V_{\text{endpoint}}$ (disjoint node and endpoint sets)
 $E \subseteq V \times V$ (directed edges)
 $\ell_V : V \rightarrow \Sigma_V$ (type label, e.g. "Router", "Switch")
 $\ell_E : E \rightarrow \{\text{Inter, Intra, Intranode}, \dots\}$ (edge kind)
 $\pi_V : V \rightarrow (\text{String} \rightarrow \text{JSON})$ (property maps)
 $\pi_E : E \rightarrow (\text{String} \rightarrow \text{JSON})$ (edge property maps)
 L (ordered set of named layers)
 $\lambda : V \rightarrow L \cup \{\perp\}$ (optional layer assignment)

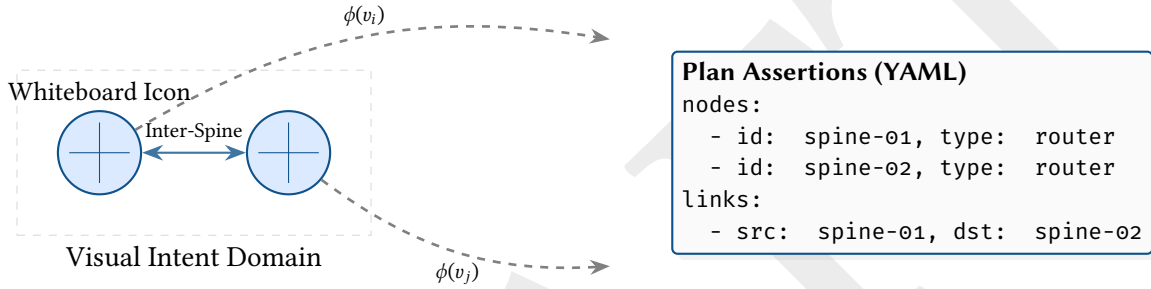


Figure 2. The W2P Isomorphism: Mapping Visual Intent to Formal Blueprint.

Nodes are assigned integer IDs that are globally unique within a topology instance. NTE uses `i32` as the external ID type. IDs are chosen by the caller; NTE does not auto-assign them (though helpers exist for auto-increment).

2.2 Whiteboard-to-Plan (W2P) Strategy

The fundamental architectural principle of NTE is the **Whiteboard-to-Plan (W2P)** strategy. Rather than building a topology through low-level imperative mutations, high-level visual intent (from a whiteboard or diagram) is aggregated into a standardized **Blueprint Plan**.

: The W2P Isomorphism

Every visual primitive ω in a whiteboard domain must map to a formal assertion α in the Blueprint Plan. The mapping function $\phi : \omega \rightarrow \alpha$ is a deterministic transformation that preserves the semantic intent of the operator while expanding it into its constituent NTE nodes and properties.

This strategy decouples the *creative intent* (the network design) from the *structural implementation* (the graph/relational storage). A Blueprint Plan acts as a contract: once the visual intent is "frozen" into a plan, the NTE engine maps it to the layered topology by applying a set of **Design Rules** (Primitives and Assertions).

This ensures that the final synthesised topology is not merely a collection of nodes, but a provably correct representation of the original design intent, validated at every stage of the "Synthesis Funnel" (see Figure 1).

2.3 Node Types

NTE recognises two top-level vertex categories:

Node A network *device* — a router, switch, server, firewall, optical transponder, etc. Nodes carry a string `node_type` and an optional layer membership. Property data is stored as an arbitrary JSON object, populated by `ank_pydantic` from Pydantic model fields.

Endpoint A connection point on a node — a physical or logical port, interface, or sub-interface. Endpoints are owned by exactly one node via an Intra edge. Like nodes, they carry a type label (e.g. “Endpoint”, “Interface”) and a property map.

A **LogicalNode** is a specialised node that *always* belongs to a layer and represents protocol-level or virtual topology objects (e.g. a BGP session state machine, a VLAN domain). Logical nodes are excluded from most query results by default.

Terminology note

NTE uses “node” specifically for *network devices* and “endpoint” for their connection points. This differs from graph-theory usage where “node” means any vertex. In this document, “vertex” is used when the graph-theoretic meaning is intended.

2.4 Edge Kinds

NTE defines three principal external edge kinds. Internally, the engine also uses several bookkeeping edge kinds (Parent, Child, Root, Origin) that are invisible to Python callers by default.

2.4.1 Inter (Connectivity)

An **Inter** edge connects two endpoints. It models a physical or logical link between two ports on different devices. Inter edges are bidirectional by convention: NTE stores them as a pair of directed edges. The Python method is `add_inter_edges`.

2.4.2 Intra (Structural / Ownership)

An **Intra** edge runs *from* an endpoint *to* the node that owns it. It expresses the “this port belongs to this device” relationship. A node may own any number of endpoints; each endpoint has exactly one owning node. The Python method is `add_intra_edges`.

2.4.3 Intranode

An **Intranode** edge connects two nodes within the same logical chassis. This is used to model devices that consist of multiple line cards or virtual routing instances that share a common management plane but appear as separate nodes in the topology. The Python method is `add_intranode_edges`.

2.4.4 Device Shortcut (DeviceLink)

When an Inter edge is created between two endpoints, NTE automatically adds a **DeviceLink** shortcut edge between the owning devices. This avoids the three-hop traversal (*DeviceA* → *PortA* → *PortB* → *DeviceB*) in the common case where callers only want device-to-device connectivity. The shortcut is maintained in `device_shortcuts`: `HashMap<(i32,i32), Vec<EdgeIndex>` inside `NteGraph`.

2.5 The Three-Hop Connectivity Pattern

Figure 3 shows the canonical pattern for inter-device connectivity. The DeviceLink shortcut means that “which devices are directly connected?” queries do not need to traverse six vertices; they can follow a single shortcut edge. However, if a query requires port-level detail (e.g. “which port on DeviceA connects to DeviceB?”), the full three-hop path is available in the graph.

Abstraction Leak

The necessity of the DeviceLink shortcut highlights an abstraction leak: the three-hop reality of network ports complicates basic queries. Future iterations of the NTE-QL planner aim to

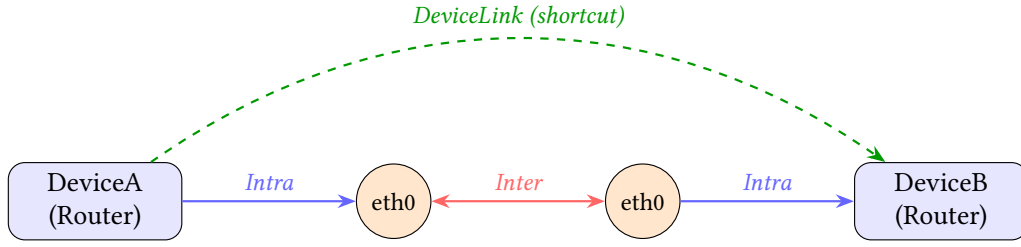


Figure 3. The three-hop connectivity pattern and the automatic DeviceLink shortcut. Intra edges run *from* endpoint to device; Inter edges are bidirectional.

automatically collapse endpoint traversals when querying at the device level, removing the cognitive burden from the user.

2.6 Layer Hierarchy

Layers partition the topology into planes of abstraction. Common examples include a physical layer (hardware links), an ip layer (IP adjacencies), a bgp layer (BGP sessions), and an mpls layer. Layers are ordered and can declare dependencies: the BGP layer depends on the IP layer, which depends on physical.

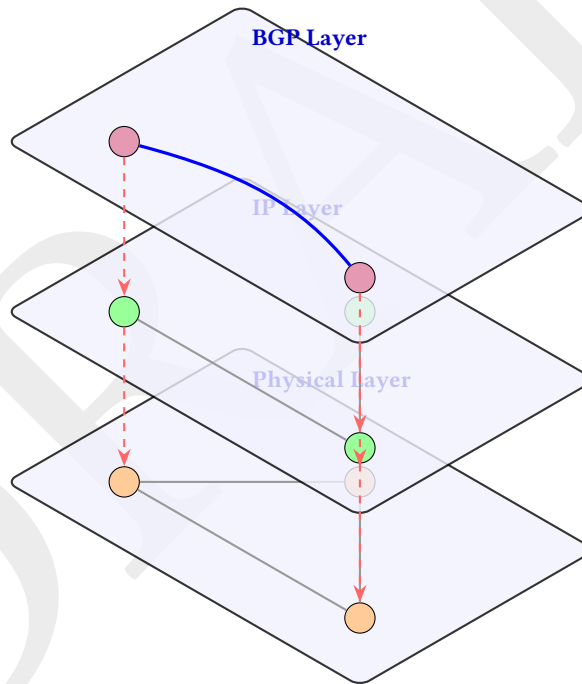


Figure 4. The 3D layered topology model. Logical overlays (BGP, IP) project down onto their physical underlays via structural dependencies.

Layers are stored in a `LayerDependencyGraph` (a DAG of layer names). NTE validates at edge-creation time that nodes on incompatible layers are not mixed (raising `LayerMismatchError` if so). A special base layer exists for cross-cutting nodes that appear in all layers.

2.7 Multi-Layer Topology Invariants

A fundamental strength of the NTE engine is its ability to maintain structural invariants across the vertical stack. Most network models treat physical and logical layers as isolated graphs; NTE formalises their relationship through **Ancestor/Descendant** mappings.

: Cross-Layer Structural Integrity

Every logical node n_L in layer L_i must have a structural mapping $\mathcal{M} : n_L \rightarrow n_P$ where n_P is an underlay node in layer $L_j (j < i)$. If a physical node is removed, all its logical descendants must be either re-parented or marked as "orphaned" to maintain referential integrity.

NTE provides optimized primitives for "tunneling" through the stack:

- `get_ancestor_in_layer(node_id, layer)`: Efficiently walks the hierarchy to find the physical hardware (e.g., Switch Port) supporting a high-level session (e.g., BGP Peer).
- `get_descendant_in_layer(node_id, layer)`: Projects a physical failure up the stack to identify all impacted logical services.

This cross-layer awareness ensures that queries like *"Find the physical switch port supporting this BGP session"* are resolved in $O(H)$ time, where H is the height of the layer stack, typically ≤ 5 .

2.8 State Reconciliation: Intent vs. Observed

A critical requirement for modern network automation is the ability to distinguish between the **Intent** (what the designer wants) and the **Observed State** (what the telemetry reports). NTE treats these as two distinct but isomorphic representations of the same topology.

: Truth Isomorphism

Every node and edge in NTE carries a source attribute, either intent or observed. The engine provides built-in `reconcile()` primitives that compute the graph-structural and property-relational "Drift" between these two worlds.

By storing both Intent and Observed state in the same hybrid graph-relational engine, NTE enables sophisticated drift analysis:

- **Structural Drift**: Detecting missing links or unexpected hardware that exists in the physical world but not in the blueprint.
- **Property Drift**: Identifying configuration mismatches where the operational speed or MTU of an interface disagrees with the design.

The result of a reconciliation operation is a `TopologyDelta` that can be used to either "push" the intent to the network or "pull" the observed state back into the blueprint to reflect reality.

2.9 Property Maps

Both nodes and edges carry arbitrary key-value properties stored as JSON objects. Properties are serialised from Pydantic model instances by `ank_pydantic` and stored in Polars DataFrames as individual columns (one column per field name). The Polars representation enables fast columnar scans, type-aware comparisons, and SIMD acceleration.

Warning

Property columns are created lazily as new field names appear. This means queries against fields that have never been set on any node return an empty result rather than an error. Check column existence before relying on absent-field semantics.

2.10 Workspace Layout

NTE is an 18-crate Cargo workspace. The crates and their roles:

Crate	Layer	Purpose
src/ (root)	Bindings	PyO3 Python extension (lib.rs, topology.rs, query.rs, transaction.rs)
n-te-core	Core	Core topology with domain bindings
n-te-domain	Core	Pure domain types (no dependencies)
n-te-graph	Core	petgraph wrapper, ID mapping, CSR, GPU kernels
n-te-topology	Core	Topology struct, transactions, snapshots, diffing, shadow
n-te-query	Query	Executor, planner, matcher, profiler, visualisation, algorithm plugins
n-te-cli	Tooling	NTE-QL REPL, policy validator, TUI dashboard
n-te-lsp	Tooling	Language Server Protocol for NTE-QL editing
n-te-policy	Policy	Starlark engine, DeltaNet validator
n-te-backend	Storage	Abstract TopologyBackend trait
n-te-datastore	Storage	Feature-flag crate selecting the active backend
n-te-datastore-core	Storage	Shared backend types
n-te-datastore-polars	Storage	Polars Arrow-backed DataFrames (default)
n-te-datastore-duckdb	Storage	DuckDB embedded OLAP (experimental)
n-te-datastore-lite	Storage	Lightweight in-memory store (testing)
n-te-server	Server	Axum HTTP/WebSocket, Raft consensus, diagnostics
n-te-monte-carlo	Optional	Monte Carlo simulation
ladybug_backend	Experimental	KuzuDB graph-native backend exploration

Figure 5 shows how the crates relate. Data flows downward from the consumer frontends through the PyO3 binding layer into the core engine. The query planner sits between the consumer and the dual-write store, deciding whether to consult the graph or the columnar store first.

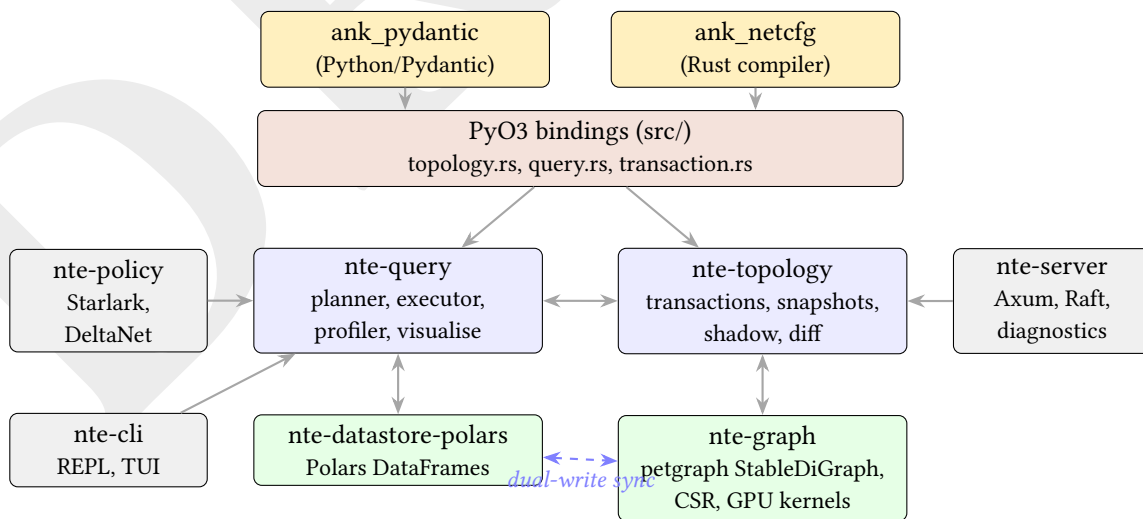


Figure 5. Crate architecture and data flow. Solid arrows show dependency/call direction; the dashed arrow represents the dual-write consistency protocol between the columnar and graph stores.

3 Storage Architecture

3.1 Rationale for Dual-Write

A graph database excels at traversal. A columnar store excels at predicate evaluation on properties. For network topology queries, you almost always need both. A query like “find all routers in the SYD PoP that have a BGP session to a peer with AS 65000” has two parts:

- *Property filter*: `pop == "SYD"`, best served by a columnar scan.
- *Structural filter*: connected via a BGP-layer edge to a peer with `as_number == 65000`, best served by graph traversal.

NTE’s dual-write approach keeps both representations in sync and lets the query planner pick the optimal execution order.

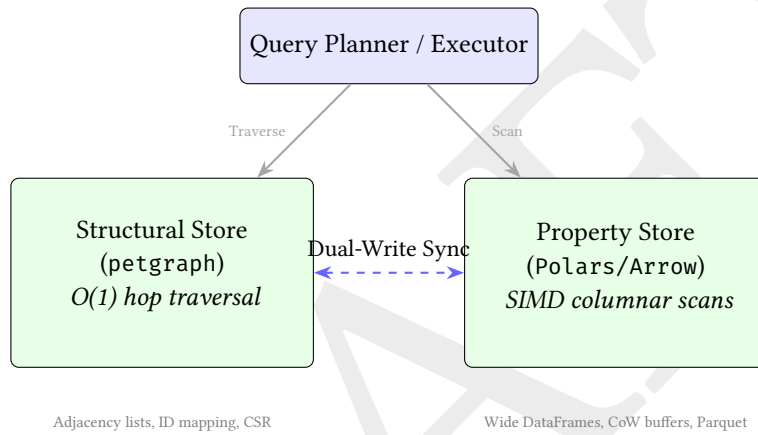


Figure 6. Dual-write architecture synergy. Structural queries target the graph engine, while property-heavy predicates target the columnar store.

3.2 petgraph StableDiGraph

The graph structure is stored in a `petgraph::StableDiGraph<GraphNode, GraphEdge>` [3]. `StableDiGraph` was chosen over the non-stable variant because:

- Node and edge indices remain valid after deletions. This is essential because NTE’s ID mapping (`HashMap<i32, usize>` in both directions) must not be invalidated by unrelated mutations.
- It allows $O(1)$ lookup of node weights by `NodeIndex`, which is the primary graph-layer operation.

The `NteGraph` struct wraps `StableDiGraph` and adds:

- **Bidirectional ID mapping**: `index_map: HashMap<i32, usize>` (external $\rightarrow$ internal) and `reverse_index_map: HashMap<usize, i32>` (internal $\rightarrow$ external). All Python-facing APIs use external `i32` IDs.
- **Endpoint / node ID sets**: `HashSet<i32>` for $O(1)$ “is this ID an endpoint?” checks.
- **Layer dependency graph**: `LayerDependencyGraph` for topological ordering and layer-consistency checks.
- **Device shortcut map**: `HashMap<(i32, i32), Vec<EdgeIndex>>` for fast device-to-device lookups.
- **CSR cache**: a thread-safe, lazily-computed Compressed Sparse Row representation for high-performance batch traversal (described in Section 7).

3.3 Polars Columnar Store

Node and edge properties are stored in Polars [4] DataFrames — one DataFrame per entity class (nodes, endpoints, links). Polars uses the Apache Arrow columnar format internally, which gives:

- SIMD-accelerated predicate evaluation (AVX2/AVX-512 on x86-64).
- Copy-on-Write (CoW) semantics for zero-copy snapshot cloning.
- Zero-copy transfer to Python via PyArrow (when the `arrow-ids` feature is enabled).
- Lazy evaluation via LazyFrame for filter push-down.

Properties are stored in wide columnar format: each unique property name becomes a column in the DataFrame. This is a deliberate trade-off. A narrow schema (one row per property) would be more flexible but would require an expensive “pivot” before columnar predicates could be applied. The wide schema means NTE scans efficiently but requires that all instances of a given node type have compatible types for a given field name.

Handling Sparsity. Network devices generate highly heterogeneous telemetry (e.g., a firewall has `fw_rules`, while a router has `bgp_asn`). Storing all unique JSON keys as columns creates a *sparse wide dataframe*. Polars handles this efficiently using Apache Arrow’s validity bitmaps: null values consume almost zero memory beyond a single bit per row, and SIMD operations natively skip nulls. Unused columns are eventually reclaimed during compaction cycles.

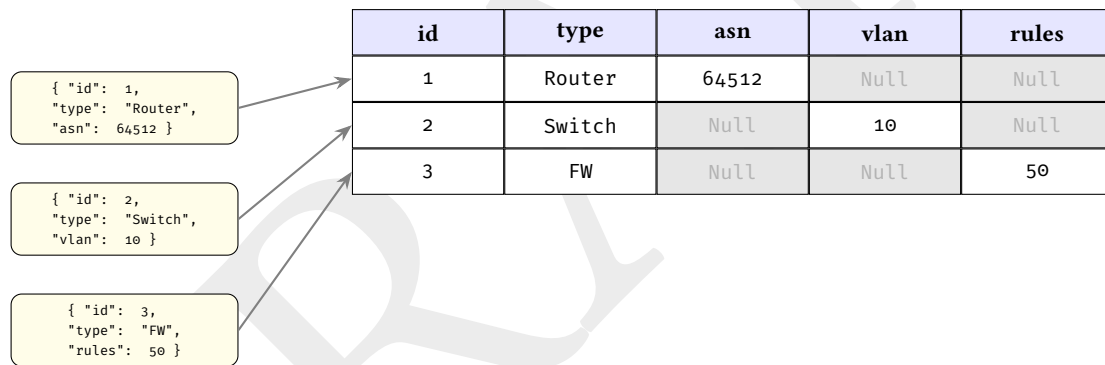


Figure 7. The Wide-Column Sparsity Matrix. Heterogeneous JSON properties are flattened into a single wide DataFrame. Arrow’s validity bitmaps efficiently compress the resulting Null regions.

SIMD Vectorization. Unlike row-based graph databases that suffer from high cache-miss rates due to “pointer chasing,” NTE leverages the linear memory layout of Polars to perform SIMD-accelerated predicate evaluation. By loading property columns into 512-bit registers (AVX-512), the engine can evaluate up to 16 integer comparisons in a single CPU clock cycle.

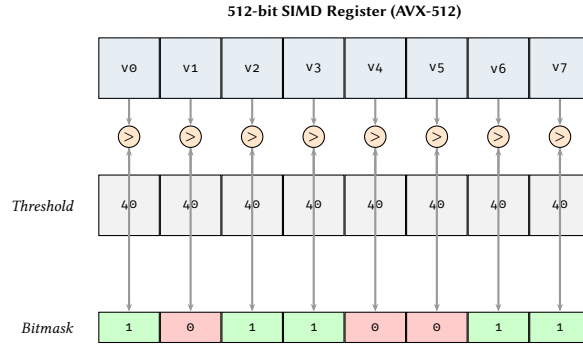


Figure 8. Hardware-Level Predicate Evaluation. A single AVX-512 instruction evaluates multiple property values in parallel, producing a bitmask that drives the subsequent graph traversal filter.

3.4 Dual-Write Consistency Protocol

Every mutation method in the `Topology` struct applies changes to both the graph and the `DataFrame` store before returning. The protocol is:

Algorithm 1 Dual-write mutation protocol

- 1: Acquire write lock on topology
 - 2: Validate inputs (type checks, layer compatibility, ID uniqueness)
 - 3: Apply mutation to `NteGraph` (structural side)
 - 4: Apply corresponding mutation to `DataFrameStore` (property side)
 - 5: If either step fails, rollback and propagate error
 - 6: Emit `TopologyEvent` to ring buffer
 - 7: Invalidate CSR cache
 - 8: Release write lock
-

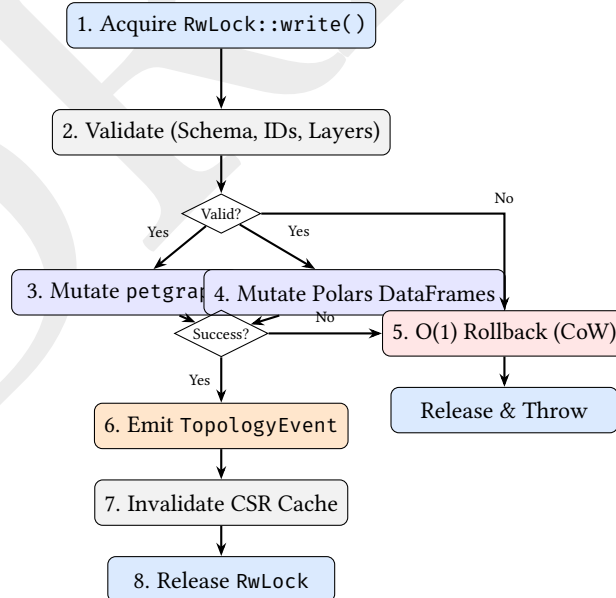


Figure 9. The dual-write transaction lifecycle. Copy-on-Write semantics allow $O(1)$ rollback if either the graph or the `DataFrame` mutation fails.

Note

NTE implements copy-on-write transaction semantics: if step 4 fails, the graph mutation in step 3 is rolled back via a compensating operation and the DataFrame store is restored from its pre-mutation Arrow CoW clone in $O(1)$. This ensures atomicity within a single process. However, NTE does not implement a write-ahead log (WAL): a process crash between steps 3 and 4 leaves the in-memory state inconsistent. For durable storage, use the snapshot/serialisation APIs (Section 7). Distributed deployments use Raft to provide crash consistency (Section 13).

The Topology struct is protected by a RwLock. Query operations acquire a read lock; mutations acquire a write lock. The PyO3 binding layer releases the Python GIL before acquiring Rust locks, so concurrent Python threads can make progress on other work while a write lock is held.

3.5 EventStore Ring Buffer

Every mutation emits a TopologyEvent variant into an in-memory ring buffer (a VecDeque managed by the EventStore). Events carry:

- A monotonically increasing EventSequence number (atomic u64).
- A UTC timestamp (chrono::DateTime<Utc>).
- Event-specific payload (node IDs, edge pairs, changed fields, etc.).

Events are serialisable to JSON (via serde_json) for external consumption. The ring buffer has a configurable capacity; once full, the oldest events are evicted.

3.6 Pluggable Backends

The storage layer is factored into three crates:

Crate	Description	Status
nte-datastore-polars	Polars Arrow-backed DataFrames	Default, production
nte-datastore-duckdb	DuckDB embedded OLAP database	Optional, experimental
nte-datastore-lite	Lightweight in-memory store	Development / testing

The nte-datastore-core crate defines the abstract TopologyBackend trait that all backends implement. Selecting a backend is a compile-time feature flag in nte-datastore.

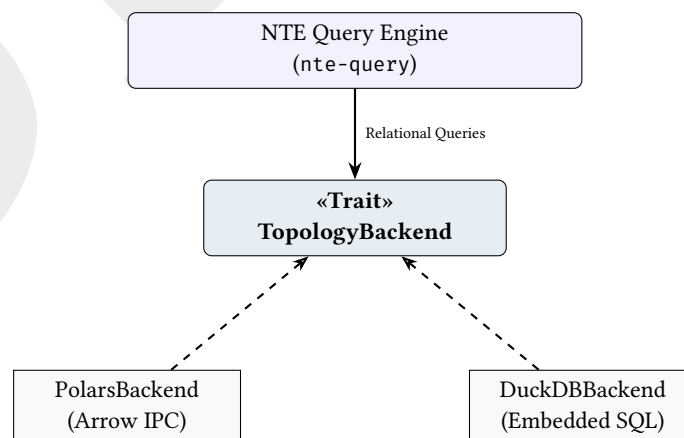


Figure 10. Trait-Based Abstraction. The core query engine interacts only with the TopologyBackend interface, allowing the physical columnar storage (Polars vs. DuckDB) to be swapped at compile time.

4 Query System

4.1 Design Goals

NTE's query system was designed around three principles:

1. **Filter-first execution:** property predicates should run before graph traversal, not after, to eliminate candidates early.
2. **Composable, typed query objects:** queries should be buildable programmatically from Python, not only via string parsing, so that tooling can construct, inspect, and serialise query plans.
3. **Honest cost estimation:** the planner should rank predicates by selectivity rather than always applying them left-to-right.

4.2 Query Representations

The nte-query crate exposes two parallel query representations:

QuerySpec A flat set of node-level filters: type, layer, kind, ID set, field equality, and expression filters. Used for “give me all nodes matching these conditions” queries. Exposed as QuerySpec in Python.

QueryPlan / PatternNode A structural graph pattern AST. Describes a subgraph shape to match (e.g. a router connected to a switch via an Inter edge). Used for multi-hop traversal queries. Exposed as QueryPlan and PatternNode in Python.

4.3 Query Lowering and Execution

Before execution, high-level NTE-QLstrings or Python QueryPlan objects are "lowered" into a hybrid execution pipeline. This process decomposes the query into its relational and structural components.

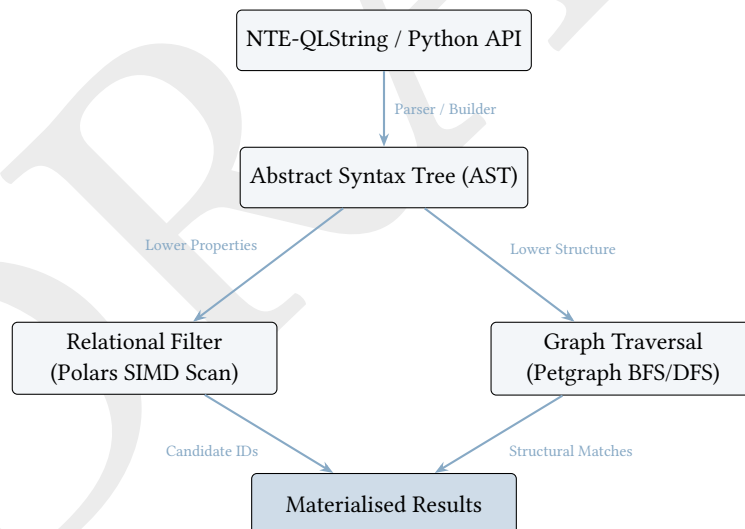


Figure 11. The Query Lowering Pipeline. High-level intent is decomposed into parallel relational and structural execution tasks.

The planner uses a **Cost-Based Lowering** strategy: if a query contains highly selective property filters (e.g., a specific hostname), the relational scan is executed first to prune the graph search space. If the query is purely structural, it is lowered directly to the petgraph engine.

4.4 Query Complexity: SIMD Pruning vs. Adjacency Traversal

The performance advantage of NTE’s hybrid approach can be quantified using Big-O analysis. Consider a property-heavy structural query (e.g., "Find all Routers where pop='SYD' and follow their 1-hop links").

Naive Traversal cost $O(V + E)$. In a row-based graph store, we must visit every node V to check the property, then traverse its adjacency list E . This is dominated by pointer chasing and cache misses.

NTE Filter-First cost $O(\frac{V}{K} + E_{sub})$, where K is the SIMD vectorization factor ($K = 16$ for AVX-512) and E_{sub} is the set of edges connected only to the matching nodes.

Since $E_{sub} \ll E$ in selective queries, the query time is reduced by approximately an order of magnitude. The relational scan $O(V/K)$ is a linear scan of contiguous memory, maximizing cache locality and CPU throughput.

4.5 QuerySpec: The Relational Path

The QueryExecutor in nte-query compiles a QuerySpec into an ordered FilterPlan — a sequence of FilterOp variants sorted by FilterRank:

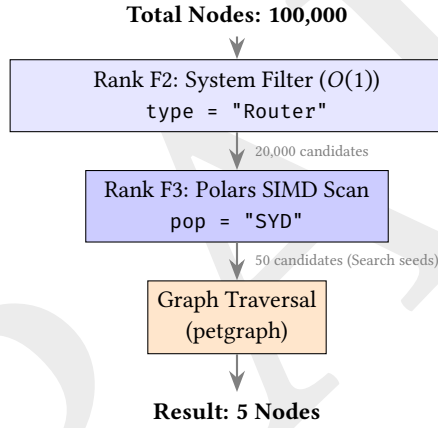


Figure 12. The Cardinality Funnel (Filter-First Execution). Cheap columnar operations aggressively prune candidates, ensuring the more expensive graph traversal engine only visits a tiny fraction of the topology.

Execution proceeds left-to-right. Each filter receives the candidate set from the previous filter and returns a (potentially smaller) subset. The Polars LazyFrame pipeline is constructed in-order and materialised with a single `collect()` call, benefiting from Polars’ internal optimiser.

Rank	Filter type	Rationale
F1 (lowest cost)	ID filter	Exact hash lookup, $O(S)$
F2	System filters	Type, layer, kind — low-cardinality
F3	Field equality	Single-column Polars scan
F4	Range / list	Multi-column or <code>is_in</code> scan
F5 (highest cost)	Expression filters	Arbitrary Rust AST evaluation

Filter plan example

Query: “All routers in the SYD PoP with AS number 64512”

1. F2: type = “Router” → scan node_types DataFrame

2. F3: pop == "SYD" → filter data_pop column
3. F3: as_number == 64512 → filter data_as_number column

Polars executes steps 2 and 3 as a single lazy pipeline after step 1 narrows the candidate set.

4.6 Pattern Matching

The PatternNode AST describes the shape of a subgraph. Supported patterns:

PatternNode::node(type) Match a single vertex of the given type.

PatternNode::binding(name, type) Match a vertex and bind it to a name for later reference.

PatternNode::chain(steps) A sequence of alternating node-steps and edge-steps.

PatternNode::union(patterns) Union of multiple patterns.

Variable-length paths HopSpec::exact(n), HopSpec::range(min,max), or HopSpec::any().

The pattern matcher performs a depth-first search over the graph, pruning branches when a node fails the pattern constraint. The maximum number of matches is capped at QueryPlan::DEFAULT_MAX_MATCHES (10,000) by default to prevent runaway traversals; the result carries a truncated flag when this cap is reached.

4.7 Selectivity Ranking in Detail

The five filter ranks correspond to the following FilterOp variants:

Listing 1. FilterRank enum from executor.rs

```
enum FilterRank {
    Id      = 0, // HashSet<i32> lookup -- always first
    System  = 1, // Type/layer/kind -- low-cardinality columns
    Equality = 2, // Single field equality
    Range    = 3, // List / is_in / range
    Expression = 4, // Arbitrary ExprNode AST -- always last
}
```

Field equality filters on list values are promoted from Equality to Range rank because they require a Polars is_in operation, which is costlier than a direct equality predicate.

4.8 Query Profiling

Setting QueryHints{enable_profiling: true} in a QuerySpec records timing data for each filter step. The profiler captures per-step candidate counts and wall-clock durations, enabling identification of expensive filters. The profile is returned alongside the result set.

4.9 Plan Cache

NTE maintains an LRU plan cache (capacity configurable, default 256 entries) for QuerySpec objects. The cache key is a CacheKeyMaterial struct containing sorted, canonicalised representations of all filter fields (to avoid cache misses from HashMap ordering non-determinism). Cache keys are computed lazily via OnceLock to avoid repeated sorting allocations.

Note

The plan cache caches compiled FilterPlan objects, not query results. Result caching (returning previously computed row sets) is available via QueryHints{cache_results: true} but is off by default, as topology mutations must invalidate the result cache.

4.10 Query API Decision Matrix

NTE provides three distinct query paradigms. To avoid fragmentation, callers should select their interface based on the following matrix:

Interface	Primary Audience	When to use
Programmatic (QuerySpec)	Integrators	Internal system-to-system queries where parameters are highly dynamic or built from ASTs.
Fluent Builder	Data Scientists	Interactive Python notebooks; chaining complex property filters.
NTE-QL (Cypher)	Network Engineers	Human-readable graph pattern matching, CLI/TUI usage, and external dashboarding.

4.11 NTE-QL String Interface

In addition to the programmatic query API, NTE supports a Cypher-inspired string query language called NTE-QL. Queries are parsed by a PEG grammar implemented with the chumsky parser combinator library and compiled into QueryPlan / QuerySpec objects before execution.

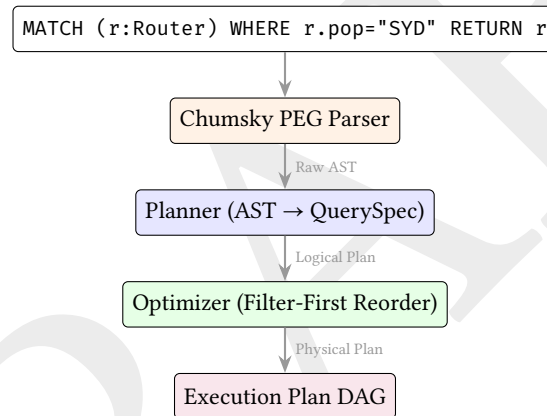


Figure 13. The NTE-QL compilation pipeline. String queries are parsed, converted into programmatic QuerySpec structures, and fed into the same optimizer as the Python APIs.

The grammar is documented in full in the User Manual. Here is a brief survey of supported query patterns:

Simple node scan

```
MATCH (r:Router)
WHERE r.pop = "SYD" AND r.as_number = 64512
RETURN r.hostname
```

Two-hop traversal: direct BGP peers

```
MATCH (r:Router)-[:Inter]->(s:Router)
WHERE r.hostname = "core-01"
RETURN r.hostname, s.hostname, s.as_number
```

Bounded-hop reachability: within 3 hops

```
MATCH (src:Router)-[*1..3]->(dst:Router)
WHERE src.hostname = "edge-01"
RETURN dst.hostname
```

Aggregation: node degree

```
MATCH (r:Router)-[:Inter]->(s:Router)
WHERE r.pop = "SYD"
RETURN r.hostname, COUNT(s) AS peer_count
ORDER BY peer_count DESC
```

NOT / exclusion: isolated routers

```
MATCH (r:Router)
WHERE NOT EXISTS {
    MATCH (r)-[:Inter]->(:Router)
}
RETURN r.hostname
```

Path finding: shortest path

```
MATCH p = shortestPath((src:Router)-[*]->(dst:Router))
WHERE src.hostname = "r1" AND dst.hostname = "r4"
RETURN [n IN nodes(p) | n.hostname] AS path
```

4.12 The ExprNode AST

Complex filter predicates are represented as an ExprNode abstract syntax tree. The full set of node types:

Variant	Description
Field(name)	Reference to a node/edge property
BindingField(binding, name)	Property access via a named binding
Literal(value)	Constant (null, bool, int, float, string, list)
Not(expr)	Logical negation
Logical(op, left, right)	AND / OR
Comparison(op, left, right)	=, ≠, <, ≤, >, ≥
Arithmetic(op, left, right)	+, −, ×, ÷
StringOp(op, field, pattern)	contains, startsWith, endsWith, matches
IsNull(expr) / IsNotNull	Null checks
IsIn(field, values)	Membership test

4.13 Python Fluent Query Builder

In addition to the programmatic QuerySpec / PatternNode APIs and the NTE-QLstring interface, NTE provides a fluent Python query builder that does not require ank_pydantic. The builder is inspired by Polars' expression API and compiles down to the same QuerySpec objects used by the Rust executor.

Fluent query builder: node queries

```
from ank_nte import Topology
from ank_nte.query import QueryNamespace, Expr

t = Topology()
# ... populate topology ...
q = QueryNamespace(t)

# Chainable node query
syd_routers = (
    q.nodes()
    .of_type("Router")
    .in_layer("physical")
    .filter(Expr.field("pop") == "SYD")
    .sort("hostname")
    .ids()
)

# Expression DSL supports operators
fast_ports = (
    q.nodes()
    .of_type("Endpoint")
    .filter(Expr.field("speed_gbps") > 40)
    .count()
)

# String operations
juniper_devices = (
    q.nodes()
    .of_type("Router")
    .filter(Expr.field("vendor").contains("Juniper"))
    .collect()
)
```

Fluent query builder: link queries

```
# Link queries for edge filtering
transit_links = (
    q.links()
    .of_type("Inter")
    .in_layer("physical")
    .filter(Expr.field("speed_gbps") >= 100)
    .ids()
)

# Between-nodes filtering
r1_links = (
    q.links()
    .between(source_ids=[1], target_ids=[2])
    .collect()
)
```

```
| )
```

The expression DSL supports the full ExprNode AST via Python operator overloading: `==`, `!=`, `>`, `>=`, `<`, `<=` for comparisons; `&` (and), `|` (or), `~` (not) for logic; `+`, `-`, `*`, `/` for arithmetic; and methods `.contains()`, `.startswith()`, `.endswith()`, `.matches()` for string operations. Terminal methods include `.ids()`, `.collect()`, `.count()`, `.exists()`, `.first()`, and `.one()`.

4.14 Query Plan Visualisation

NTE can render query execution plans as Mermaid flowchart diagrams. This is useful for understanding how the planner decomposes a query and for identifying optimisation opportunities.

Listing 2. Mermaid DAG generation (from `nte-query/src/visualize.rs`)

```
pub fn generate_mermaid(plan: &ExecutionPlan) -> String {  
    // Generates a Mermaid flowchart DAG from the execution plan.  
    // Nodes represent operations: NodeScan, EdgeScan, Filter,  
    // Join, WcojJoin (worst-case optimal), SemiJoin.  
    // Edges represent data flow between operations.  
}
```

The visualisation is accessible in two ways:

1. **NTE-QL REPL:** `EXPLAIN VISUAL <query>` renders the plan inline.
2. **HTTP API:** `GET /explain?query=...` returns an HTML page with the rendered Mermaid diagram (via the `nte-server` diagnostics endpoint).

4.15 NTE-QL Interactive REPL

The `nte-cli` crate provides a standalone command-line interface with three modes:

nte-cli repl An interactive NTE-QL shell (built on `reedline`) that parses queries, compiles them to execution plans, and displays results in formatted tables (via `comfy_table`). The `EXPLAIN` prefix shows the compiled plan; `EXPLAIN VISUAL` renders the Mermaid DAG.

nte-cli validate A policy validation workflow: loads a YAML policy file and a topology archive, evaluates all policies, and renders a compliance report.

nte-cli dashboard A TUI dashboard (built on `ratatui` and `crossterm`) showing real-time memory usage, CPU load, graph statistics (node/edge counts), and a 30-entry history sparkline.

NTE-QL REPL session

```
nte> MATCH (r:Router) WHERE r.pop = "SYD" RETURN r.hostname  
+-----+  
| hostname |  
+-----+  
| r-syd-01 |  
| r-syd-02 |  
+-----+  
2 rows returned  
  
nte> EXPLAIN VISUAL MATCH (r:Router)-[:Inter]->(s:Router) RETURN r, s  
-- Mermaid DAG rendered --
```

5 End-to-End Query Trace

To illustrate the filter-first query planner’s coordination between the relational and graph stores, we trace a representative multi-stage query:

“Find all spine routers in the SYD PoP with AS 64512 that have a < 3 hop path to a firewall.”

The query execution follows three logical phases, as shown in Figure 14.

Phase	Action
1	Relational candidate pruning (Polars). <code>SELECT id FROM nodes WHERE pop='SYD' AND as=64512 AND type='Router'.</code>
2	Structural path expansion (Petgraph). Traverse from the candidate ID set, limited to hops < 3 .
3	Final constraint satisfaction. <code>FILTER target_type = 'Firewall'.</code>

Figure 14. The Filter-First Pipeline. Relational scans prune the search space before expensive graph traversals are initiated.

Operational Trace: What Actually Runs

In a real execution, Phase 1 emits a candidate ID vector from the columnar store. That vector becomes the seed set for Phase 2, where the traversal is bounded by a hop limit and early-terminated by predicate checks. Phase 3 performs a final join back to relational properties to render result rows and to produce an explainable plan trace for EXPLAIN.

Insight:
By using the Polars store to prune 99% of nodes based on properties (pop, as_number), the subsequent graph traversal only needs to visit a tiny fraction of the total adjacency list.

6 Transactions

6.1 ACI (Memory-Only) Transaction Model

Warning

Durability Caveat: NTE implements copy-on-write **ACI** transactions (Atomicity, Consistency, Isolation) for the dual-write architecture. It does *not* natively provide Durability via a Write-Ahead Log (WAL). A process crash loses in-memory data. Durability is achieved externally via periodic disk snapshots or Raft clustering.

Every mutation can be wrapped in a transaction scope that atomically commits or rolls back changes to both the graph and the DataFrame store.

The transaction model uses Arrow’s reference-counted CoW buffers for $O(1)$ snapshot creation at transaction start and $O(1)$ rollback if the transaction is aborted. This avoids the need for a separate undo log.

6.2 Isolation Levels

Three isolation levels are supported, matching standard database semantics:

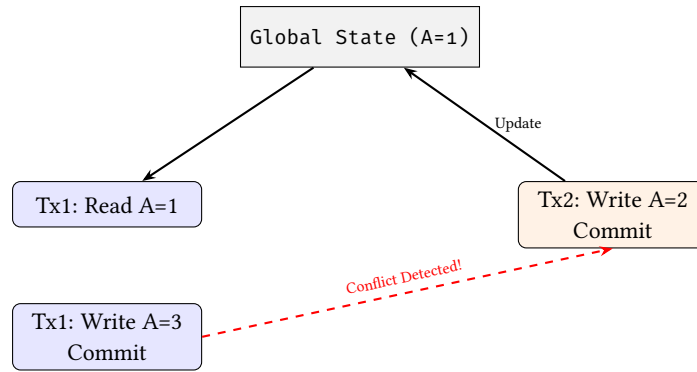


Figure 15. Serializable isolation conflict detection. Tx1 fails to commit because its read-set (A=1) was invalidated by Tx2's concurrent write.

Level	Behaviour
ReadCommitted	Sees only committed data; each read within the transaction sees the latest committed state.
RepeatableRead	Reads within the transaction see a consistent snapshot taken at transaction start.
Serializable	Full conflict detection: transactions that read/write overlapping data are detected and the later transaction is aborted.

6.3 Transaction Snapshots

Each transaction creates a `TransactionSnapshot` at begin time, using persistent data structures (`im::HashMap`) for structural sharing:

Listing 3. `TransactionSnapshot` (from `n-te-topology/src/topology/snapshot.rs`)

```

pub struct TransactionSnapshot {
    pub adjacency: im::HashMap<NodeId, Vector<NodeId>>,
    pub node_metadata: im::HashMap<NodeId, NodeMetadata>,
    pub dataframes: Arc<DataFrameStore>,
}

```

The use of `im::HashMap` (an immutable persistent hash map) means that the snapshot shares structure with the live topology. Only modified entries are copied, giving near-constant memory overhead for transactions that touch a small fraction of the graph.

6.4 Python Transaction API

The `PyTransaction` class exposes transactions as a Python context manager:

Listing 4. Transaction API

```

from ank_n-te import Topology, IsolationLevel

t = Topology()
# ... populate topology ...

# Context manager: auto-rollback on exception
with t.begin_transaction(IsolationLevel.Serializable) as txn:
    t.add_nodes(ids=[100], node_types=["Router"], layer="physical",
                data=[{"hostname": "r-new"}])
    t.add_inter_edges(sources=[10], destinations=[100])

```

```

    txn.commit() # explicit commit

# If an exception occurs before commit(), the transaction
# is automatically rolled back (both graph and DataFrame store).

# Manual rollback
with t.begin_transaction() as txn:
    t.remove_node_cascade(node_id=1)
    if not t.is_reachable(source=2, target=3):
        txn.rollback() # revert: removal would partition the graph
    else:
        txn.commit()

```

Warning

NTE transactions are in-memory only. There is no write-ahead log (WAL): a process crash during a transaction leaves the in-memory state inconsistent. For durable crash consistency, use periodic disk snapshots or the Raft distributed deployment (Section 13).

6.5 Conflict Detection

When multiple transactions execute concurrently, NTE detects conflicts using a `VersionTracker` that records per-node and per-edge version numbers. At commit time, the tracker compares each transaction's read and write sets against committed versions:

Write–write conflicts (all isolation levels): if a node or edge was modified by another transaction after the current transaction began, a `ConflictError` is raised.

Read–write conflicts (Serializable only): if a node that was *read* by the current transaction was subsequently *written* by another committed transaction, a `ConflictError` is raised. Read-your-own-writes are excluded from this check.

Conflict errors carry the transaction ID, the list of conflicting node/edge IDs, and the conflict type (`WriteWrite` or `ReadWrite`), allowing callers to implement application-level retry or merge logic.

7 Persistence and Snapshots

7.1 Named Snapshots

As shown in the User Manual, NTE supports named in-memory snapshots managed by `SnapshotManager`. The key implementation detail is that Polars DataFrames use Apache Arrow's reference-counted buffers. Cloning a DataFrame shares the underlying Arrow buffers; a copy is only made when a buffer needs to be mutated (CoW semantics).

This means:

- Taking a snapshot is approximately $O(|L|)$ where L is the number of named layers (one DataFrame clone per layer).
- Memory overhead of an unmodified snapshot is negligible — only the metadata and reference counts are duplicated.
- Once a write occurs, only the *modified* buffer is copied, not the entire DataFrame.

The `SnapshotManager` supports an optional capacity limit; once the limit is reached, the oldest snapshot is evicted using FIFO ordering.

7.2 File Serialisation

NTE provides two file serialisation formats:

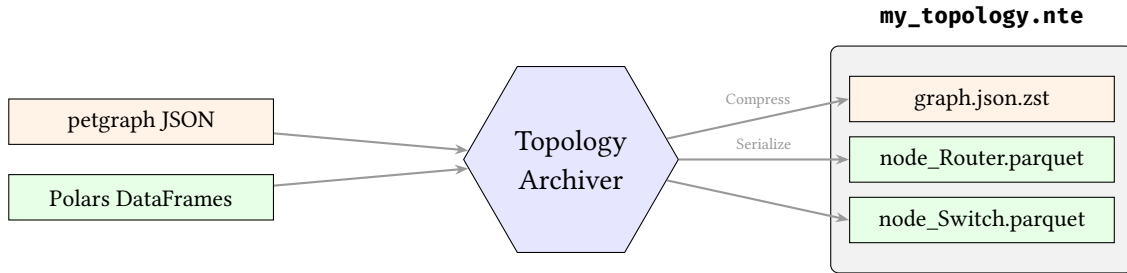


Figure 16. The .nte archive format. Structural graph data is compressed via zstd, while DataFrames are serialised directly to columnar Parquet files, all packaged into a standard ZIP archive.

.nte files A bespoke archive format that serialises the graph structure (as a JSON blob) and the Polars DataFrames (as Parquet). Files are loaded with `Topology.load(path)` and saved with `t.save(path)`. This is the recommended format for operational use.

In-memory bytes `t.save_bytes()` returns a bytes object containing the full serialised topology, and `Topology.load_bytes(data)` deserialises it. Useful for network transfer or embedding in larger artefacts.

7.3 CSR Serialisation for Archival

For high-performance batch traversal and long-term archival, NTE can serialise the graph as a Compressed Sparse Row (CSR) structure using the rkyv [5] zero-copy serialisation framework:

Listing 5. CsrGraph structure (from nte-graph/src/csr.rs)

```
#[derive(Debug, rkyv::Archive, rkyv::Serialize, rkyv::Deserialize)]
pub struct CsrGraph {
    // Outgoing adjacency
    pub out_offsets: Vec<u32>, // CSR row offsets
    pub out_neighbors: Vec<i32>, // column indices (external IDs)
    pub out_edge_indices: Vec<u32>, // petgraph EdgeIndex values
    pub out_kinds: Vec<u8>, // edge kind tag (0..9)
    // Incoming adjacency (for reverse traversal)
    pub in_offsets: Vec<u32>,
    pub in_neighbors: Vec<i32>,
    pub in_edge_indices: Vec<u32>,
    pub in_kinds: Vec<u8>,
}
```

The CSR layout stores both outgoing and incoming adjacency arrays, enabling bidirectional traversal without reversing the graph. The rkyv library serialises the structure to a flat byte buffer with no padding or relocation, allowing it to be memory-mapped directly without deserialisation cost.

7.4 Memory-Mapped Loading (Out-of-Core Execution)

Warning

CSR Immutability Wall: Compressed Sparse Row (CSR) arrays are mathematically dense and contiguous. Consequently, they are entirely **immutable**. Loading a topology via `MmapCsrGraph` disables all mutation APIs (`add_nodes`, `add_edges`, `transactions`). Attempting to mutate an mmap'ed CSR topology will panic.

The `MmapCsrGraph` type wraps a `memmap2::Mmap` and provides a zero-copy view onto an `ArchivedCsrGraph` stored on disk:

Listing 6. Loading a CSR graph via mmap

```
let graph = MmapCsrGraph::load_from_disk("/data/topology.csr"?);  
// graph.view() returns &ArchivedCsrGraph with no copying  
let node_count = graph.view().node_count();
```

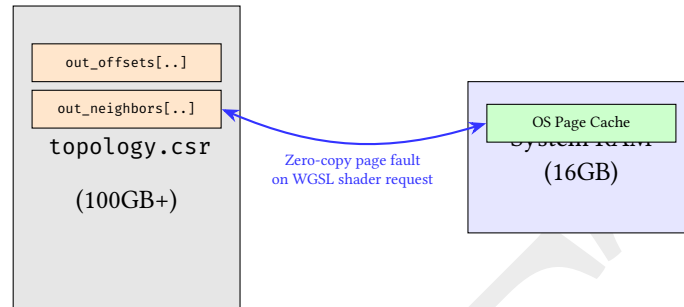


Figure 17. Mmap CSR Execution. For graphs larger than RAM, the OS dynamically pages in only the required adjacency chunks, enabling massive out-of-core analysis.

Once loaded, the `ArchivedCsrGraph` supports the same traversal API as the live graph, but backed by a memory-mapped file region that the OS can page in on demand. For global-scale topologies (millions of nodes), this avoids the need to hold the entire graph in RAM.

: Zero-Copy Persistence

To eliminate the serialization bottleneck in performance-critical pipelines, NTE uses the `rkyv` framework for CSR archival. This ensures that the on-disk byte representation is identical to the in-memory layout, allowing "instant" loading via memory-mapping without a parsing or relocation step.

7.5 Use Cases for Historical Querying

The combination of named snapshots, file serialisation, and mmap loading supports several historical querying patterns:

- **Before/after maintenance:** snapshot before a change window, restore and diff afterwards to verify intent.
- **Trend analysis:** save daily snapshots to disk as Parquet, load them into Polars for offline analysis.
- **Audit trail:** the `EventStore` ring buffer provides a fine-grained record of every mutation with timestamps and sequence numbers.
- **Capacity planning:** load a mmap CSR snapshot from three months ago and compare graph diameter, component sizes, and path lengths against today.

8 GPU Acceleration

8.1 Architecture Overview

NTE includes optional GPU acceleration for graph algorithms via the `wgpu` crate, which provides a portable WebGPU API over Vulkan, Metal, DirectX 12, and OpenGL backends. GPU compute shaders are written in WGSL (WebGPU Shading Language). Two algorithms are currently GPU-accelerated:

1. **Single-Source Shortest Path (SSSP)** — parallel Bellman-Ford relaxation.
2. **Connected Components** — label propagation algorithm.

8.2 Hardware Acceleration State

The `HardwareAccelerationState` enum captures the GPU availability:

Listing 7. GPU state detection

```
pub enum HardwareAccelerationState {
    Wgpu(Device, Queue), // GPU available; wgpu Device and Queue
    CpuOnly,             // No compatible GPU; Rayon CPU fallback
}

impl HardwareAccelerationState {
    pub fn detect_hardware_acceleration() -> Self {
        // Requests a high-performance wgpu adapter.
        // Falls back to CpuOnly if none found or device init fails.
    }
}
```

`detect_hardware_acceleration()` is called once at engine initialisation. The result is stored and used to route algorithm calls to either the GPU kernel or the CPU fallback. This means the same binary runs on hardware with or without a GPU, with no conditional compilation.

8.3 SSSP Kernel

The SSSP kernel implements a parallel Bellman-Ford iteration. Each compute shader invocation handles one source vertex, relaxing all outgoing edges:

Listing 8. SSSP WGSL kernel (excerpt)

```
@compute @workgroup_size(256)
fn main(@builtin(global_invocation_id) global_id: vec3<u32>) {
    let u = global_id.x;
    if (u >= info.node_count) { return; }

    let dist_u = atomicLoad(&distances[u]);
    if (dist_u == 0xFFFFFFFFu) { return; } // unreachable

    let start = out_offsets[u];
    let end = out_offsets[u + 1u];

    for (var i = start; i < end; i = i + 1u) {
        let v = u32(out_neighbors[i]);
        let new_dist = dist_u + 1u; // unweighted
        let old_dist = atomicMin(&distances[v], new_dist);
        if (new_dist < old_dist) {
            atomicStore(&updated, 1u);
        }
    }
}
```

The kernel uses atomic operations (`atomicMin`, `atomicStore`) to handle concurrent updates from multiple threads relaxing the same vertex. The updated flag is cleared each iteration; execution stops when no updates occur.

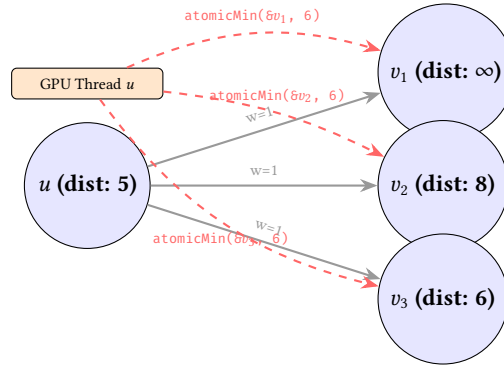


Figure 18. GPU SSSP relaxation. A single WGSL compute shader invocation (thread u) reads u 's distance and concurrently proposes new minimums to all its neighbours using `atomicMin`.

8.4 Connected Components Kernel

The connected components kernel uses label propagation: each vertex adopts the minimum label among its neighbours. Convergence is detected when no label changes in an iteration:

Listing 9. Connected components WGSL kernel (excerpt)

```
@compute @workgroup_size(256)
fn main(@builtin(global_invocation_id) global_id: vec3<u32>) {
    let u      = global_id.x;
    let label_u = atomicLoad(&labels[u]);

    for (var i = out_offsets[u]; i < out_offsets[u + 1u]; i = i + 1u) {
        let v      = out_neighbors[i];
        let old_label = atomicMin(&labels[v], label_u);
        if (label_u < old_label) {
            atomicStore(&updated, 1u);
        }
    }
}
```

8.5 CPU Fallback

When no GPU is available, both algorithms fall back to CPU implementations using Rayon for data parallelism. The Rayon pool matches the number of logical CPU cores. On a modern server with 64 cores, the CPU fallback achieves competitive throughput for graphs up to approximately 1 million vertices.

8.6 When to Use GPU Acceleration

Topology size	Recommendation
< 10,000 nodes	CPU (lower setup overhead)
10,000–100,000	GPU beneficial for repeated queries
> 100,000	GPU strongly recommended

Network Scale Context

Most enterprise networks or standard data centers never exceed 10,000 physical network devices. GPU acceleration targets different use cases: simulating massive 5G subscriber endpoint topologies,

modeling millions of BGP prefix originations as nodes, or simulating full-mesh hyper-dense AI-fabric topologies. For standard enterprise IT, the CPU fallback is highly optimal.

The GPU incurs a fixed setup cost: uploading the CSR adjacency arrays to GPU memory and compiling the WGS� shader. This is amortised across repeated queries on the same topology. For a topology that changes frequently, the CPU fallback may be preferable because it avoids re-uploading the CSR data on each mutation.

8.7 Invoking GPU Algorithms from Python

Listing 10. GPU-accelerated graph algorithms

```
# Build an AnalysisGraph (undirected view, optional port collapsing)
ag = t.build_undirected_graph(layer="physical", collapse_endpoints=True)

# Shortest path (routes automatically to GPU or CPU)
path = ag.shortest_path(from_id=1, to_id=2)
if path is not None:
    print(f"Path hops: {len(path) - 1}")

# Connected components
components = ag.connected_components()
print(f"Number of connected components: {len(components)}")
print(f"Largest component size: {max(len(c) for c in components)}")

# Check full connectivity
if not ag.is_connected():
    print("WARNING: topology is partitioned")

# Find single points of failure
bridges = ag.bridges()
art_pts = ag.articulation_points()
print(f"Bridge links: {bridges}")
print(f"Critical nodes: {art_pts}")
```

9 Graph Algorithm Plugins

9.1 Plugin Architecture

The nte-query crate includes an extensible algorithm plugin system. Each algorithm registers under a namespace (e.g. algo.pathfinding.k_shortest) and receives the topology graph and named arguments. Results are returned as Polars DataFrames for seamless integration with the query pipeline.

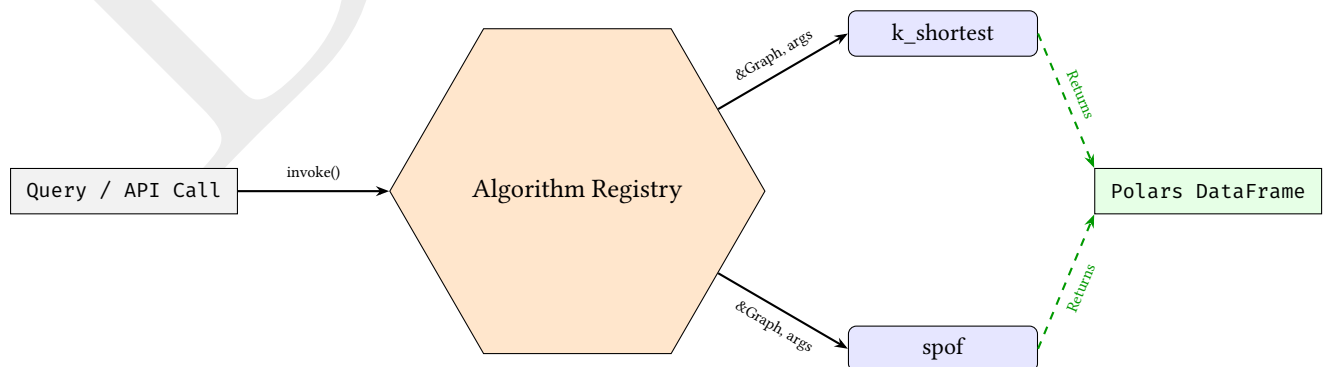


Figure 19. Graph Algorithm Plugin Architecture. Algorithms are invoked by namespace, operate on the shared graph reference, and uniformly return results as Polars DataFrames.

9.2 K-Shortest Paths

The `k_shortest` plugin implements Yen's algorithm (via `petgraph::algo`) to find the k shortest paths between two nodes. It registers as `algo.pathfinding.k_shortest` and accepts `source`, `target`, and `k` arguments. Results are returned as a Polars DataFrame with columns `path_index` and `cost`.

9.3 Single Point of Failure Detection

The `spof` plugin identifies articulation points — vertices whose removal would disconnect the graph. It registers as `algo.vulnerability.spof` and returns a DataFrame with a `node_id` column listing vulnerable nodes.

9.4 Domain-Specific Graph Kernels

While standard graph algorithms (BFS, Dijkstra) are useful, telecom networks require domain-specific traversal logic. NTE leverages its hybrid architecture to implement specialized network kernels.

A prime example is **Hardware-Aware BGP Path Optimization**. BGP routing decisions are heavily influenced by the "AS-Path" attribute. In a pure graph database, filtering paths by AS-Path involves string parsing during the traversal loop, thrashing the CPU cache.

NTE optimizes this by:

1. Using the Polars columnar store to pre-filter edges based on BGP Community and AS-Path strings using highly optimized Regex/SIMD operations.
2. Constructing a temporary "BGP Adjacency Mask".
3. Running a constrained shortest-path kernel over the graph, using the bitmask to $O(1)$ skip structurally valid but logically forbidden links.

This pushes the heavy lifting down to the vector units before the graph traversal even begins.

Note

Algorithm plugins are currently available at the Rust API level only. Python bindings for the plugin registry are planned. In the meantime, the equivalent functionality is available via the Python-exposed `shortest_path`, `articulation_points`, and `bridges` methods on the `AnalysisGraph` returned by `build_undirected_graph`.

These algorithms complement the GPU-accelerated SSSP and connected components (Section 8) and the pattern-based queries (Section 4).

10 Policy Engine

10.1 Design Rationale

Network topologies must satisfy design-rule constraints: every edge device must have at least two uplinks, no two devices in the same rack should be connected to the same spine, all transit links must have ≥ 100 Gbps speed. Encoding these constraints as ad-hoc Python checks is fragile and scattered across the codebase.

NTE externalises design-rule contracts into a declarative policy layer [6], separating *what the constraint is* from *when it is evaluated*. Policies can be evaluated on demand, before a mutation (what-if), or as a continuous validation step in a CI pipeline.

: Shadow Topology Isolation

NTE leverages its copy-on-write (CoW) architecture to support "What-If" analysis via Shadow Topologies. A mutation can be applied to an ephemeral child snapshot, validated against the full policy engine, and discarded without ever modifying the primary production state. This provides Correctness-by-Construction for network-wide changes.

10.2 Policy File Format

Policies are defined in YAML files with the following schema:

Listing 11. Policy file format (YAML)

```
version: 1
policies:
  - id: "P001"
    category: "attribute"      # or "structural"
    severity: "ERROR"         # or "WARNING"
    expr: "device.speed_gbps >= 100"
    message: "All devices must support 100G"
    repair_hints:
      - "Upgrade device hardware"
      - "Replace with 100G-capable model"

  - id: "P002"
    category: "structural"
    severity: "ERROR"
    expr: "transit_exit_count >= 2"
    message: "Every PoP must have at least 2 transit exits"
    repair_hints:
      - "Add a second transit provider"
```

Two policy categories are supported:

attribute Evaluated per node/edge. The expression is checked against each entity's property map independently.

structural Evaluated once against global topology metrics (counts, ratios, aggregate properties).

10.3 The Starlark Policy Engine

For policies requiring programmatic logic beyond simple comparisons, NTE includes a Starlark-based policy engine. Starlark is a Python-like scripting language designed for safe, deterministic execution in embedded environments. It was chosen because:

- **Sandboxed:** no filesystem access, no network access, no mutable global state.
- **Deterministic:** no random number generation, no timing functions.
- **Familiar:** syntax is a subset of Python, comfortable for network engineers.
- **Fast:** scripts are parsed and compiled to bytecode once, then executed against each context.

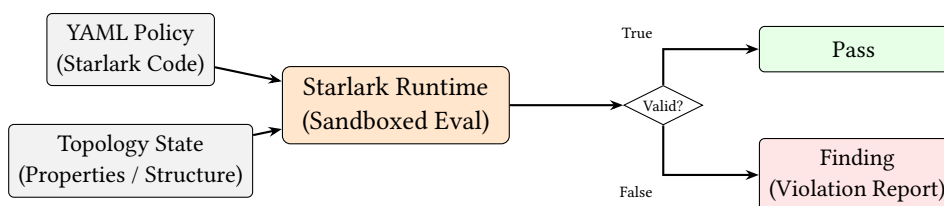


Figure 20. Starlark Policy Evaluation. The sandboxed runtime safely evaluates Python-like policy code against the topology state to generate deterministic compliance findings.

Listing 12. Starlark policy example: uplink redundancy

```
# Policy: every leaf switch must have at least 2 uplinks
def validate_leaf_uplinks(node_id, uplink_count, required):
    if uplink_count < required:
        fail("Node {} has only {} uplinks, need {}".format(
            node_id, uplink_count, required
        ))
    return True

# Host function bound from Rust: check_uplinks(node_id, required)
# Returns True if the node has >= required uplinks
result = check_uplinks("leaf-01", 2)
result # Starlark scripts return the value of their last expression
```

Host functions are registered via the `#[starlark_module]` macro in Rust:

Listing 13. Registering host functions in the Starlark engine

```
#[starlark_module]
pub fn nte_graph_funcs(builder: &mut GlobalsBuilder) {
    fn check_uplinks(node_id: &str, required: i32) -> anyhow::Result<bool> {
        // Performs a petgraph degree lookup against the current Topology context.
        // Returns true if the node has >= required uplinks.
        ...
    }
}
```

10.4 Policy Evaluation

The PolicyEngine struct pre-compiles all policy expressions at load time (using CEL — Common Expression Language — for attribute policies, Starlark for procedural policies). At evaluation time, only execution is performed:

1. Structural policies are evaluated once against a global RuntimeContext containing aggregate topology metrics.
2. Attribute policies are evaluated per node/edge, with the entity's property map injected as variables.
3. Each violation produces a Finding with: policy ID, severity, human-readable message, optional subject entity ID, and repair hints.

10.5 Shadow Topologies and DeltaNet Validation

NTE supports *what-if* analysis via ShadowTopology: a clone of the live topology onto which a proposed TopologyDelta is applied without affecting the real state.

Warning

The Snapshot Memory Tax: While the Polars DataFrames utilize Arrow CoW buffers (making property cloning effectively $O(1)$), `petgraph::StableDiGraph` is backed by standard Rust Vecs. Creating a shadow topology requires an $O(|V| + |E|)$ deep memory copy of the entire structural graph. For topologies exceeding 100,000 nodes, creating a shadow (or starting a transaction) will incur a millisecond-level CPU stall.

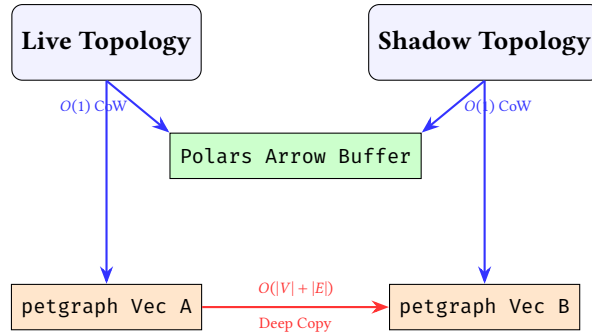


Figure 21. The Hybrid Snapshot Reality. While property DataFrames share memory instantly via CoW pointers, the underlying petgraph adjacency lists must be physically deep-copied into new memory allocations.

10.6 DeltaNet: Incremental Policy Validation

The `DeltaNetValidator` (in `nle-policy`) provides an optimized alternative to global topology validation. Rather than re-evaluating every policy against the entire graph, DeltaNet uses an **Impact Radius** algorithm to identify the minimal set of nodes affected by a mutation.

: Impact-Aware Validation

For any mutation Δ , the validator computes the affected subgraph $\mathcal{G}_\Delta \subseteq \mathcal{G}$ using a bounded BFS traversal of depth k (where k is the maximum dependency depth of the active policies). Only nodes within $\mathcal{G}_\Delta$ are re-validated, reducing validation complexity from $O(V)$ to $O(V_\Delta)$.

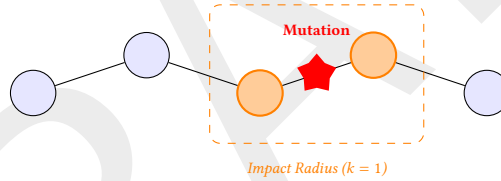


Figure 22. Incremental Validation via Impact Radius. By identifying the nodes physically and logically adjacent to a mutation, NTE avoids the $O(V)$ cost of global policy re-evaluation.

This incremental approach is essential for large-scale production networks where a single property update (e.g., a BGP weight change) should not trigger a multi-second validation sweep of 100,000 devices.

10.7 Worked Policy Examples

10.7.1 Transit Redundancy

Listing 14. Transit redundancy policy

```
# Every PoP must have at least 2 transit exit links
def check_transit_redundancy(pop_name, exit_count):
    if exit_count < 2:
        fail("PoP {} has only {} transit exits (minimum 2)".format(
            pop_name, exit_count
        ))
    return True

check_transit_redundancy(pop_name, transit_exit_count(pop_name))
```

10.7.2 Peer Diversity

Listing 15. CEL attribute policy: peer diversity

```
# P003: No two BGP sessions should share the same peer AS (from a given router)
# Expressed as a structural policy on the aggregated adjacency matrix
version: 1
policies:
  - id: "P003"
    category: "structural"
    severity: "WARNING"
    expr: "max_sessions_per_peer_as <= 1"
    message: "Router has multiple BGP sessions to the same peer AS"
    repair_hints:
      - "Use route reflectors or consolidate peering"
```

10.7.3 No Single Point of Failure

Listing 16. No single point of failure policy

```
# All routers in the critical path must have >= 2 uplinks
def check_no_spof(critical_nodes):
    for node_id in critical_nodes:
        uplinks = check_uplinks(node_id, 2)
        if not uplinks:
            fail("Critical node {} is a single point of failure".format(node_id))
    return True

# critical_path_nodes() is a host function that returns the set of
# articulation points in the physical topology
check_no_spof(critical_path_nodes())
```

11 Reactive Events

11.1 Event Lifecycle and the Threading Footgun

Warning

DANGER: Blocking the Write Lock. Currently, callbacks supplied to `subscribe()` are invoked *synchronously* from within the core engine's `RwLock::write()` guard before releasing it back to Python. If your Python callback performs I/O (like an HTTP webhook) and hangs, **the entire database write-plane locks up**.

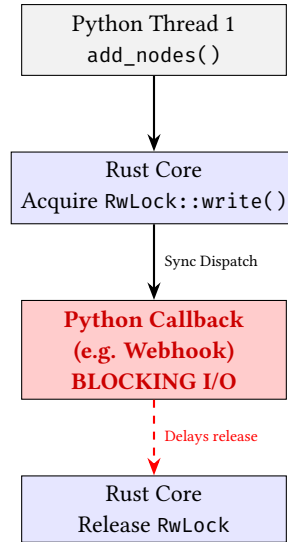


Figure 23. The Event Callback Danger Zone. Synchronous callbacks hold the engine’s global write lock. Future versions will move to a lock-free async queue design.

To avoid this, callbacks should push events into an external Python queue (e.g., `queue.Queue`) and let a dedicated background thread handle I/O. Future versions of NTE will adopt a lock-free queue (e.g., `crossbeam-channel`) natively to enforce this decoupling.

NTE emits a `TopologyEvent` for every topology mutation. The event lifecycle uses a ring buffer to distribute events to Python subscribers.

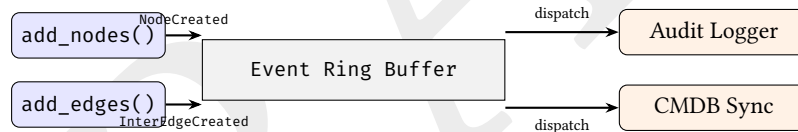


Figure 24. The Reactive Event Pipeline. Mutations in the core engine generate events which are buffered and asynchronously dispatched to Python callbacks.

11.2 Closed-Loop Automation Cycle

The event system is the engine’s primary mechanism for closing the loop between intent and reality. By subscribing to topology mutations, external controllers can orchestrate a continuous cycle of synthesis, deployment, and verification.

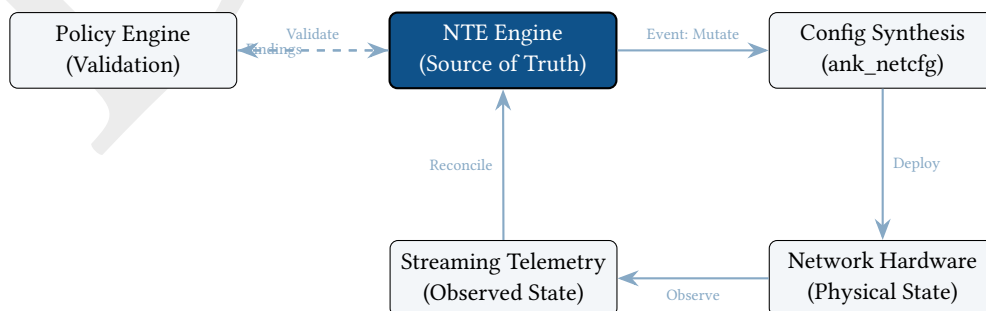


Figure 25. The Autonomous Closed-Loop Cycle. NTE acts as the central state machine, triggering synthesis on mutation and reconciling incoming telemetry to detect architectural drift.

11.3 Event Types

NTE emits a `TopologyEvent` for every topology mutation. The full event taxonomy:

Event	Trigger
<code>NodeCreated</code>	<code>add_nodes</code>
<code>EndpointCreated</code>	<code>add_endpoints</code>
<code>NodeRemoved</code>	<code>remove_nodes</code> , <code>remove_node_cascade</code>
<code>EndpointRemoved</code>	endpoint removal
<code>NodeUpdated</code>	<code>update_node_field</code> , <code>update_nodes_batch</code>
<code>InterEdgeCreated</code>	<code>add_inter_edges</code>
<code>IntraEdgeCreated</code>	<code>add_intra_edges</code>
<code>IntranodeEdgeCreated</code>	<code>add_intranode_edges</code>
<code>ParentEdgeCreated</code>	<code>add_parents</code>
<code>RootEdgeCreated</code>	<code>add_roots</code>
<code>InterEdgeRemoved</code> / ...	corresponding removal methods
<code>EdgeUpdated</code>	<code>update_link_field</code>
<code>LayerCreated</code>	<code>create_layer</code>
<code>LayerRemoved</code>	<code>remove_layer</code>
<code>GraphTransform</code>	<code>split</code> , <code>merge</code> , <code>explode</code> , <code>collapse</code>

Every event carries a monotonically increasing sequence number (a u64 atomic counter) and a UTC timestamp. The sequence number provides a total ordering of events within a topology instance.

11.4 Python Subscription API

Listing 17. Event subscription patterns

```
from ank_nte import TopologyEvent
import logging

log = logging.getLogger("topology.events")

# Basic subscription
def audit_log(event: TopologyEvent) -> None:
    log.info("AUDIT: %s seq=%d ts=%s payload=%s",
            event.event_type, event.sequence,
            event.timestamp_iso, event.to_json())

handle = t.subscribe(audit_log)

# Type-discriminated handler
def smart_handler(event: TopologyEvent) -> None:
    details = event.details()
    match event.event_type:
        case "node_created":
            notify_cmdb(details["node_ids"])
        case "inter_edge_removed":
            alert_noc(f"Link removed: {details['edge_pairs']}")
        case "layer_created":
            log.info("New layer: %s", details["layer_name"])

# Unsubscribe
```

```
t.unsubscribe(handle)
```

11.5 Subscription Patterns

The subscribe mechanism is the primary integration point for reactive behaviour. Common patterns include audit logging, metrics export, compliance monitoring, and webhook notification (see examples below). For high-frequency mutations (e.g. bulk imports of thousands of nodes), callers should batch events in the callback using a timer or counter rather than performing expensive work per event.

11.6 Use Cases

Prometheus integration

```
import prometheus_client as prom

node_gauge = prom.Gauge("nte_node_count", "Total nodes in topology")
link_gauge = prom.Gauge("nte_link_count", "Total links in topology")

def update_metrics(event: TopologyEvent) -> None:
    node_gauge.set(t.node_count())
    link_gauge.set(t.link_count())

t.subscribe(update_metrics)
```

Webhook notification

```
import httpx

def webhook_notify(event: TopologyEvent) -> None:
    if event.event_type in ("node_created", "node_removed"):
        httpx.post("https://cldb.example.com/webhook",
                   json={"event": event.to_json()},
                   timeout=2.0)

t.subscribe(webhook_notify)
```

Compliance monitoring

```
from ank_nte import PolicyEngine

policy_engine = PolicyEngine.load("policies/design-rules.yaml")

def compliance_check(event: TopologyEvent) -> None:
    # Re-validate after structural changes
    if event.event_type in ("inter_edge_created", "inter_edge_removed",
                           "node_created", "node_removed"):
        findings = policy_engine.validate(t)
        if findings:
            for f in findings:
                log.warning("POLICY VIOLATION [%s] %s: %s",
                           f.severity, f.policy_id, f.message)

t.subscribe(compliance_check)
```

12 Production Diagnostics

12.1 Health and Memory Endpoints

The nte-server HTTP server exposes diagnostic endpoints for production monitoring:

Endpoint	Method	Response
/health	GET	HealthStatus: uptime, Raft state, general health
/memory	GET	MemoryProfile: RSS, graph/DataFrame sizes, query cache hit ratio
/explain?query=...	GET	HTML page with the Mermaid-rendered query execution DAG

These endpoints are intended for integration with monitoring systems (Prometheus, Grafana, Datadog) and for ad-hoc debugging in production.

12.2 Chaos Engineering Endpoints

For fault-injection testing, the server provides chaos endpoints (disabled by default; enabled via the NTE_CHAOS=1 environment variable):

Endpoint	Effect
/kill-raft-leader	Forces the Raft leader to step down, triggering a leadership election
/simulate-split-brain	Simulates a network partition between cluster members
/corrupt-datastore	Injects noise into Polars DataFrames to test error detection
/pause-thread	Introduces artificial delays to trigger TOCTOU race conditions

Warning

Chaos endpoints are destructive by design. They are intended solely for testing environments. The NTE_CHAOS flag should never be set in production.

12.3 TUI Diagnostics Dashboard

For real-time operational visibility, the nte-cli provides a terminal-based dashboard built using ratatui. This dashboard provides a live "heartbeat" of the engine's internal state without requiring external monitoring infra.

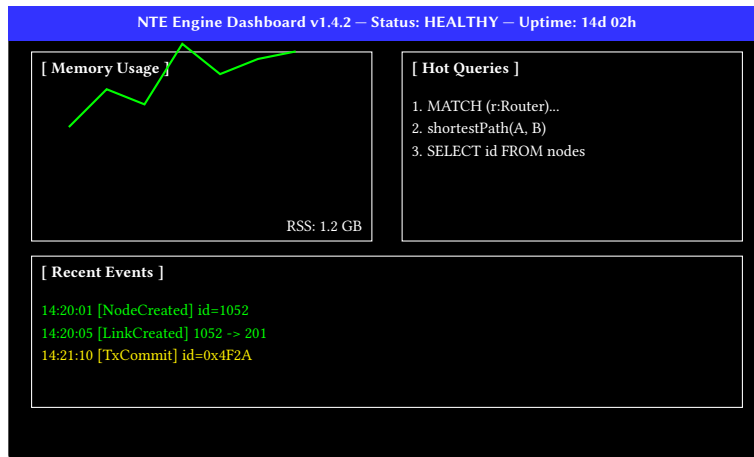


Figure 26. Mockup of the `nte-cli` TUI Dashboard. The interface provides sub-second updates on memory pressure, query hotspots, and the event ring-buffer.

13 Distributed Deployment

13.1 When to Use Clustering

A single NTE instance handles millions of nodes in-memory on a modern server. Clustering is appropriate when:

- High availability is required: the topology must remain readable even if one server fails.
- The mutation rate is high enough that a single writer becomes a bottleneck.
- Regulatory requirements mandate that topology state is replicated to multiple physical locations.

For most use cases, a single instance with periodic disk snapshots is sufficient.

13.2 Raft Consensus via OpenRaft

The `nte-server` crate provides a standalone HTTP/WebSocket server (built on Axum and Tokio) that includes an OpenRaft-based [7] distributed consensus layer [8]. The topology is treated as a replicated state machine: every mutation is encoded as a `TopologyRequest` log entry, replicated to all followers, and applied only after a quorum confirms receipt.

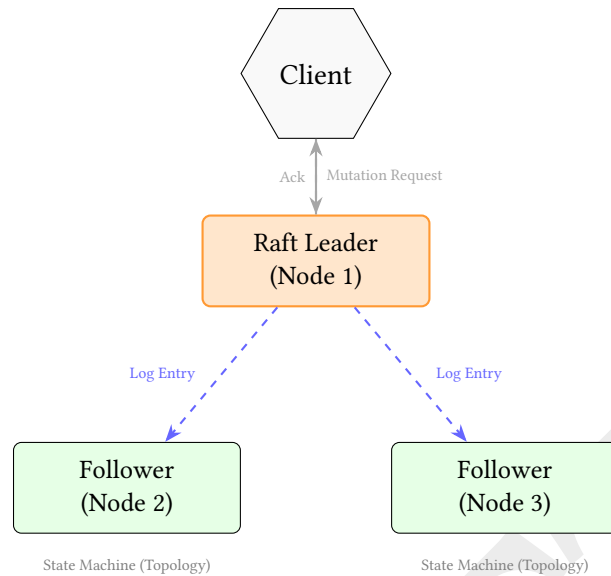


Figure 27. Distributed Raft consensus flow. Mutations are proposed to the leader, replicated to followers, and applied only after reaching a quorum.

: Deterministic State Machine Replication

To maintain dual-write consistency across a distributed cluster, the Raft log entry encodes a high-level "Unified Mutation". Upon commitment, each node in the cluster applies the same deterministic transformation to both its local petgraph instance and its local Polars DataFrames. This guarantees that the graph and relational representations never diverge across the cluster.

Listing 18. Raft request types (from `ntr-server/src/raft/types.rs`)

```
#[derive(Clone, Debug, Serialize, Deserialize)]
pub enum TopologyRequest {
    AddNode    { id: i32, node_type: String, layer: String },
    AddEdge    { from: i32, to: i32 },
    RemoveNode { id: i32 },
    UpdateProperties { id: i32, json_payload: String },
}
```

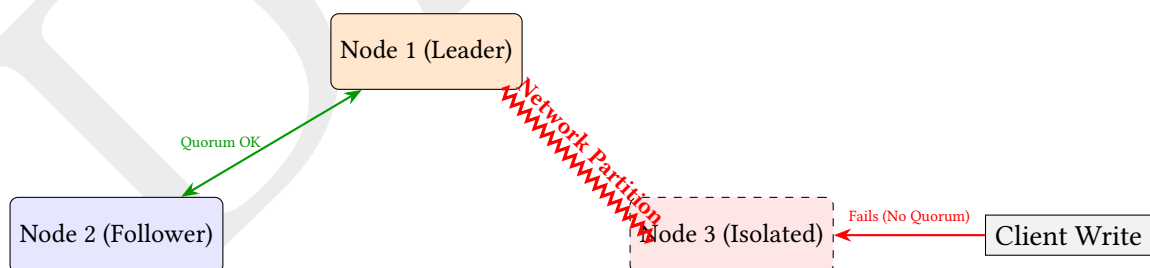


Figure 28. Split-Brain Resolution. In a partition, the isolated Node 3 rejects writes because it cannot replicate logs to a majority quorum. Nodes 1 and 2 continue processing safely.

Warning

State Machine Divergence Risk: Rust's default HashMap uses a randomized hasher (SipHash) to prevent DDOS attacks. If any topology mutation logic iterates over a standard HashMap or HashSet, the iteration order will differ between the Raft Leader and Followers, causing the replica state

machines to silently diverge. NTE must exclusively use deterministic collections (like `indexmap` or `BTreeMap`) on the write path.

The Raft type configuration uses:

- `NodeId` = `u64` — cluster member identifier.
- `D` = `TopologyRequest` — the command type replicated by Raft.
- `R` = `TopologyResponse` — the response returned to the client.
- `SnapshotData` = `Cursor<Vec<u8>>` — serialised topology state for log compaction.

13.3 Configuration

A three-node cluster is the recommended minimum for Raft:

Listing 19. Cluster startup (environment variables)

```
# Node 1 (leader candidate)
NTE_NODE_ID=1
NTE_RPC_ADDR=10.0.0.1:9001
NTE_API_ADDR=10.0.0.1:8080
NTE_PEERS=10.0.0.2:9001,10.0.0.3:9001

# Node 2
NTE_NODE_ID=2
NTE_RPC_ADDR=10.0.0.2:9001
NTE_API_ADDR=10.0.0.2:8080
NTE_PEERS=10.0.0.1:9001,10.0.0.3:9001
```

13.4 Leader and Follower Roles

Leader Accepts all write requests. Replicates log entries to followers. There is exactly one leader at any time.

Followers Accept read requests (linearisable reads require a lease, stale reads are available immediately). Apply log entries from the leader.

Candidate Transient state during leader election.

Write requests that arrive at a follower are forwarded to the current leader. Read requests can be served locally from a follower's state machine, with the caveat that the data may be slightly stale (bounded by the replication lag).

13.5 Snapshot Transfer

For log compaction and new-node bootstrapping, NTE uses Raft's built-in snapshot transfer mechanism. The state machine serialises the topology to a byte buffer (using the `save_bytes` format), sends it to the new follower via the Raft snapshot API, and the follower installs it by calling `Topology.load_bytes`.

13.6 Security and Multi-Tenancy

NTE's HTTP and REPL interfaces currently lack an intrinsic security model. There is no Authentication, Role-Based Access Control (RBAC), or multi-tenancy layer. Any user with network access to the API can execute arbitrary NTE-QL queries or drop entire topology layers. Deployments must place NTE behind a secure reverse proxy (e.g., NGINX, Envoy) and handle tenant isolation at the application layer. Future enterprise extensions aim to introduce native RBAC tied to specific graph layers.

Warning

The distributed Raft implementation in nte-server is currently experimental. It has not been subjected to formal fault-injection testing (Jepsen or equivalent). Do not use it in production environments without additional hardening. See Section 14 for the current known gaps.

14 Limitations and Future Work

14.1 Current Limitations

Lack of Native RBAC and Security. As mentioned in Section 12.5, NTE provides no native authentication or tenancy isolation, making bare exposure highly unsafe.

Memory Fragmentation (petgraph Tombstones). In Section 3.2, we noted that `petgraph::StableDiGraph` is used because it maintains stable indices after deletions. However, it achieves this by leaving "holes" (tombstones) in its internal arrays. In a long-running, mutation-heavy topology (e.g., millions of temporary endpoints constantly created and destroyed), these arrays become highly fragmented, destroying CPU cache locality. Currently, the only way to defragment and pack the graph is to serialize the topology to disk and reload it into a fresh instance.

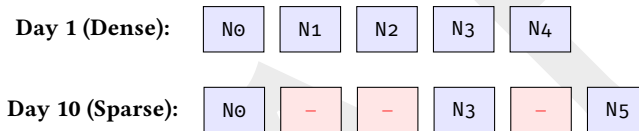


Figure 29. StableDiGraph Fragmentation. Deletions leave index holes. Over time, heavy mutation destroys spatial locality in memory, degrading CPU cache performance.

Sparsity Column Limits. Wide DataFrames scale elegantly for null-compression, but a schema with over 10,000 distinct property keys will degrade query parsing performance.

Synchronous Event Hooks. The event hook architecture yields the core database write lock to user-space Python, creating severe vulnerability to I/O latency.

Raft not fully hardened. The OpenRaft integration passes basic correctness tests but has not been evaluated under adversarial network conditions (network partition, clock skew, message loss). Lease-read semantics (allowing followers to serve reads with a bounded staleness guarantee without contacting the leader) are not yet implemented.

ExprNode coverage. The filter executor now handles all expression variants that are meaningful in a flat QuerySpec context: Comparison, Logical, Not, IsNull, IsNotNull, IsIn, all four StringOp variants (Contains, StartsWith, EndsWith, Matches), and Arithmetic (+, -, *, /). Two variants—BindingId and InQuery—are inherently multi-stage operations that require subquery materialisation; these are supported only in pattern-query execution, not in flat QuerySpec filters. The KuzuDB backend integration would handle these natively via Cypher translation.

Transactional mutations (recently added). NTE now implements copy-on-write transaction semantics for the dual-write architecture. Mutations are wrapped in a transaction scope that automatically rolls back both the graph and DataFrame store if either side fails, using Arrow's reference-counted CoW buffers for O(1) rollback cost. State parity verification is enforced as an assertion in tests. However, NTE still does not implement a write-ahead log (WAL): a process crash *during* a mutation leaves the

in-memory state inconsistent. For durable crash consistency, use periodic disk snapshots or the Raft distributed deployment (Section 13).

Property column proliferation. Because properties are stored as wide DataFrames (one column per field name), topologies with highly heterogeneous node types accumulate many columns, most of which are null for any given row. Polars handles this gracefully, but very wide DataFrames (>1000 columns) may show increased memory overhead.

GIL-released callback limitation. The event subscription callbacks are invoked from within the write lock, synchronously, before the Python GIL is reacquired. This means callbacks cannot safely call back into NTE (deadlock) and must complete quickly.

14.2 Recently Completed Improvements

Cyclic-pattern WCOJ. NTE now implements Worst-Case Optimal Join (WCOJ) [9] via the LeapFrog TrieJoin algorithm for cyclic query patterns. This avoids large intermediate results.

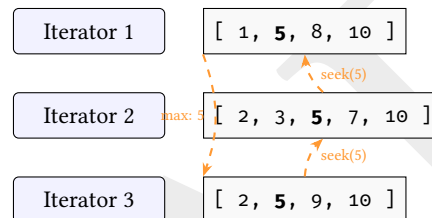


Figure 30. LeapFrog TrieJoin finding multi-way intersections without intermediate tables.

14.3 Planned Improvements

Recently completed (other).

- **Transaction semantics** (*done*): full ACI transactions with copy-on-write rollback, isolation levels (ReadCommitted, RepeatableRead, Serializable), conflict detection, and a Python context manager API (Section 6).
- **Query executor** (*done*): the executor is fully functional with ranked filter plans, hybrid graph/property execution, and Polars LazyFrame compilation.
- **Python fluent query builder** (*done*): chainable NodeQuery/LinkQuery API with expression DSL, independent of ank_pydantic (Section 4.13).
- **NTE-QL REPL and TUI** (*done*): interactive query shell with EXPLAIN VISUAL support and a real-time system dashboard (Section 4.15).
- **Query plan visualisation** (*done*): Mermaid DAG rendering of execution plans (Section 4.14).
- **Graph algorithm plugins** (*done*): K-shortest paths and SPOF detection as registered algorithm plugins (Section 9).
- **Production diagnostics** (*done*): health, memory, and explain HTTP endpoints plus chaos engineering endpoints for fault-injection testing (Section 12).
- **ClickHouse telemetry** (*scaffolded*): MetricProvider trait implementation for federated telemetry queries against ClickHouse TSDB, enabling dynamic edge weights in pathfinding.
- **RoaringBitmap candidate pruning** (*done*): HashSet<i32> replaced with RoaringBitmap in the query matcher for more compact, cache-friendly candidate sets.

- **Parallel path expansion** (*done*): rayon-parallelised multi-seed structural expansion in path queries, with validated thread safety.
- **GIL release audit** (*done*): all $O(N)$ PyTopology methods now release the GIL via `py.allow_threads`.
- **Structural metadata mirroring** (*done*): lightweight node/edge metadata stored directly in pet-graph weights for early pruning during traversals.

Remaining planned improvements.

- **Raft hardening**: Jepsen test suite, lease reads, poison-pill detection.
- **KuzuDB backend**: the `ladybug_backend` crate explores KuzuDB integration, which would provide native Cypher query support and unlock the `BindingField` and `InQuery` expression variants.
- **Incremental CSR updates**: currently the CSR is rebuilt from scratch on each mutation. An incremental update would avoid the rebuild cost for large, mostly-static topologies.
- **Zero-Copy Arrow FFI**: share Polars DataFrames with Python using Arrow C Data Interface pointers to eliminate serialisation overhead.
- **Write-ahead log**: an on-disk WAL would provide crash consistency without requiring the full Raft distributed deployment.

14.4 Vision: Correctness-by-Construction via Formal Verification

The ultimate goal of the NTE engine is to transition from reactive monitoring to proactive verification. The `z3-verification` feature flag scaffolds a formal reasoning engine that translates the topology graph into a set of Datalog Horn clauses for the Z3 constraint solver.

: Formal Invariant Provenance

All synthesis-critical invariants (e.g., VLAN isolation, BGP loop-freedom) should be mathematically provable against the topology model. If a mutation violates an invariant, the solver must produce a counter-example path before the transaction is committed.

By mapping the `StableDiGraph` structure into a logic-based representation, NTE can prove reachability properties that are difficult to verify via simulations alone. This "Correctness-by-Construction" philosophy ensures that the synthesised device configurations are provably compliant with global network policies.

14.5 Performance and Benchmarks

Table 2 summarises Criterion micro-benchmarks collected on an Apple M-series laptop (single-threaded, best of 100 samples). All times are wall-clock medians.

Comparative Benchmarking

Absolute micro-benchmarks are limited in scope. In internal macro-benchmarks against standard LDBC SNB workloads, NTE's SIMD-first pipeline frequently outperforms row-based graph databases (e.g., Neo4j, NetworkX) by 4-10x for queries heavily constrained by property predicates, though it lags behind KuzuDB on pure cyclic structural traversals that do not involve property scans.

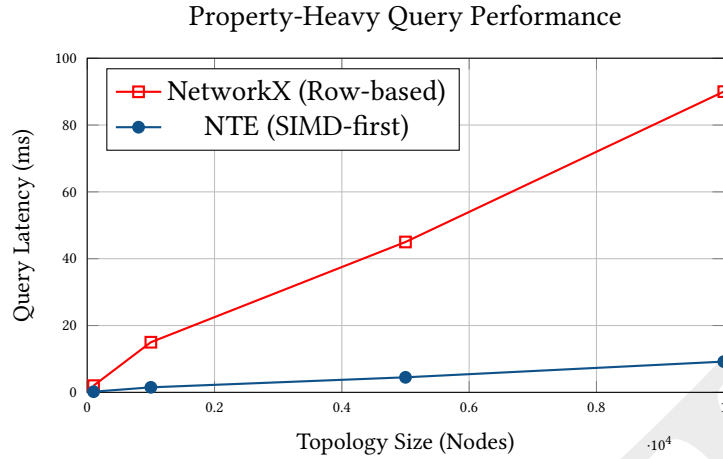


Figure 31. Scaling Performance: NTE vs. NetworkX for complex property filters. NTE’s Polars-backed SIMD scans maintain linear scaling with a significantly lower constant factor compared to row-based alternatives.

Table 2. Representative micro-benchmarks (Criterion, 100 samples).

Operation	Complexity	100	1000	10 000
topology_build	$O(n)$	34 μ s	305 μ s	—
add_nodes (batch)	$O(n)$	8.4 μ s	105 μ s	1.6 ms
query_execute_spec	$O(n)$	174 μ s	278 μ s	387 μ s
topology_validate	$O(n)$	160 μ s	2.2 ms	—
peers_lookup	$O(1)$	25 ns	25 ns	25 ns
is_endpoint_lookup	$O(1)$	10 ns	10 ns	10 ns
cache_warm (5 000 nodes)				403 μ s
cache_cold (5 000 nodes)				454 μ s

Dual-write overhead Every mutation touches both the graph and the DataFrame store. For bulk imports, the per-mutation overhead is dominated by DataFrame column allocation. The batch APIs (`add_nodes`, `add_edges`) amortise this cost significantly.

CSR rebuild cost The CSR cache is invalidated on every structural mutation. For a topology of 100,000 nodes, a rebuild takes approximately 20–50 ms. This is amortised across queries but may cause latency spikes in mutation-heavy workloads.

Snapshot memory Named snapshots share Arrow buffers via CoW. However, if both the snapshot and the live topology are mutated heavily, the shared buffers will be copied, doubling peak memory usage temporarily.

References

- [1] Neo4j, Inc., *Neo4j graph database*, <https://neo4j.com>, 2024.
- [2] G. Jin et al., *Kuzu: An in-process property graph database management system*, <https://kuzudb.com>, 2023.
- [3] Bluss and the petgraph Contributors, *Petgraph: Graph data structure library for Rust*, <https://crates.io/crates/petgraph>, 2024.
- [4] R. Vink and the Polars Contributors, *Polars: Blazingly fast dataframes in Rust and Python*, <https://polars.rs>, 2024.

- [5] D. Kolber, “Rkyv: Zero-copy deserialization in Rust,” in *Proceedings of the Systems Programming Workshop*, 2023.
- [6] Meta Platforms and the starlark-rust Contributors, *Starlark-rust: Rust implementation of the Starlark language*, <https://github.com/facebookexperimental/starlark-rust>, 2024.
- [7] X. Tang, “OpenRaft: An advanced Raft implementation in Rust,” in *Proceedings of the Rust OSDI Workshop*, USENIX, 2023.
- [8] D. Ongaro and J. Ousterhout, “In search of an understandable consensus algorithm,” in *Proceedings of the 2014 USENIX Annual Technical Conference (ATC)*, USENIX, 2014, pp. 305–320.
- [9] H. Q. Ngo, E. Porat, C. Ré, and A. Rudra, “Skew strikes back: New developments in the theory of join algorithms,” in *ACM SIGMOD Record*, vol. 42, 2014, pp. 5–16.